顧客情報システム共通クラス

表紙

項目	内容
文書名	基本設計書
システム名	
機能ID	COM-048
画面名	顧客情報システム共通クラス
会社名	
作成日	

概要

概要説明

顧客情報システム共通クラスの入力・表示・要求処理を行う。

情報	内容
対象種別	common

情報	内容
開始契機	呼出元からの要求
処理対象	N2_SIGHT.B顧客基本表、受給有無テーブル、年金1_LK、年金種類テーブル、年金__削除明細、店名テーブル、性別テーブル、裁定請求テーブル

項目一覧

No	グループ	項目名	種類	入出力	DB連携項目	候補取得元	必須	形式・桁	説明

項目定義数：0項目。表示順と反復行の定義数を区別する。

イベント仕様

CifComon

処理概要

顧客情報システム共通クラスで「CifComon」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。

getWareki

処理概要

顧客情報システム共通クラスで「getWareki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「seireki.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：「seireki.length() が異なる 10」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1.1-別.1. 条件不成立時：「条件17を満たす（原本条件は補足説明を参照）、かつ条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

1.1-別.1.2. 条件成立時：対象データを削除する。

1.1-別.1.3. 条件成立時：対象データを削除する。

1.1-別.1.4. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1-別.1-別.1. 条件不成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理23：詳細仕様参照）。
2. 繰返し条件「条件24を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：「条件25を満たす（原本条件は補足説明を参照）、かつ条件26を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1.1. 条件成立時：関連情報を取得・設定する（処理27：詳細仕様参照）。

1. 「gengoTable[i][0].equals("不明")」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理28：詳細仕様参照）。
2. 関連情報を取得・設定する（処理29：詳細仕様参照）。
3. 関連情報を取得・設定する（処理30：詳細仕様参照）。
4. 「条件31を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

10.1. 条件成立時：関連情報を取得・設定する（処理32：詳細仕様参照）。

1. 「日」を設定する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
seireki.length() が異なる 8・seireki.length() が異なる 10	処理の異常を通知し、この経路を終了する
seireki.length() が異なる 8・NOT (seireki.length() が異なる 10)・条件33を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件34を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件35を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件36を満たす（原本条件は補足説明を参照）・gengoTable[i][0].equals("不明")	処理の異常を通知し、この経路を終了する
条件37を満たす（原本条件は補足説明を参照）・NOT (gengoTable[i][0].equals("不明"))	処理結果を呼出元へ返す

getWarekiHenkan

処理概要

顧客情報システム共通クラスで「getWarekiHenkan」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または(seireki.length() が異なる 8)」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 繰返し条件「条件14を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：「条件15を満たす（原本条件は補足説明を参照）、かつ条件16を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

1. 「gengoTable[i][0].equals("不明")」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理18：詳細仕様参照）。
2. 関連情報を取得・設定する（処理19：詳細仕様参照）。
3. 関連情報を取得・設定する（処理20：詳細仕様参照）。
4. 「条件21を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

9.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 「月」を設定する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または(seireki.length() が異なる 8)	処理結果を呼出元へ返す

到達条件	結果
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・条件23を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・gengoTable[i][0].equals("不明")	処理の異常を通知し、この経路を終了する
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・NOT (gengoTable[i][0].equals("不明"))	処理結果を呼出元へ返す

get49Wareki

処理概要

顧客情報システム共通クラスで「get49Wareki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「eDate.length() が等しい 10」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：「条件7を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1.1. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

2.1-別.1. 条件不成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2-別.1. 条件不成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSimpleWareki

処理概要

顧客情報システム共通クラスで「getSimpleWareki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 対象データを削除する。
3. 対象データを削除する。
4. 対象データを削除する。
5. 対象データを削除する。
6. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSimpleGengoWareki

処理概要

顧客情報システム共通クラスで「getSimpleGengoWareki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理8：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2-別.1. 条件不成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

2-別.2. 条件不成立時：対象データを削除する。

2-別.3. 条件不成立時：対象データを削除する。

2-別.4. 条件不成立時：対象データを削除する。

2-別.5. 条件不成立時：「sbDate.substring(0,2).equals("平成")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5.1. 条件成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

2-別.5-別.1. 条件不成立時：「sbDate.substring(0,2).equals("昭和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-別.5-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("大正")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

2-別.5-別.1-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("明治")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

2-別.5-別.1-別.1-別.1-別.1. 条件不成立時：対象データを削除する。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getToday

処理概要

顧客情報システム共通クラスで「getToday」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getTime

処理概要

顧客情報システム共通クラスで「getTime」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目」を設定する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getLastMonthDate

処理概要

顧客情報システム共通クラスで「getLastMonthDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 関連情報を取得・設定する（処理12：詳細仕様参照）。
4. 関連情報を取得・設定する（処理13：詳細仕様参照）。
5. 関連情報を取得・設定する（処理14：詳細仕様参照）。
6. 関連情報を取得・設定する（処理15：詳細仕様参照）。
7. 関連情報を取得・設定する（処理16：詳細仕様参照）。
8. 関連情報を取得・設定する（処理17：詳細仕様参照）。
9. 関連情報を取得・設定する（処理18：詳細仕様参照）。
10. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

CifComonRead

処理概要

顧客情報システム共通クラスで「CifComonRead」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 関連情報を取得・設定する（処理12：詳細仕様参照）。
4. 関連情報を取得・設定する（処理13：詳細仕様参照）。
5. 関連情報を取得・設定する（処理14：詳細仕様参照）。
6. 関連情報を取得・設定する（処理15：詳細仕様参照）。
7. 関連情報を取得・設定する（処理16：詳細仕様参照）。
8. 繰返し条件「(iLen = cISR.read(cBuf)) が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

8.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

8.2. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

9.1. 条件成立時：使用した接続・処理資源を解放する。

    1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

CifComonHTMLRead

処理概要

顧客情報システム共通クラスで「CifComonHTMLRead」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 関連情報を取得・設定する（処理12：詳細仕様参照）。
4. 関連情報を取得・設定する（処理13：詳細仕様参照）。
5. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

5.1. 通常処理：関連情報を取得・設定する（処理14：詳細仕様参照）。

5.2. 通常処理：関連情報を取得・設定する（処理15：詳細仕様参照）。

5.3. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.4. 通常処理：「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.4.1. 条件成立時：新しいデータを登録する。

5.5. 通常処理：関連情報を取得・設定する（処理18：詳細仕様参照）。

5-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

5-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

結果・例外時の扱い

到達条件	結果
条件19を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
この終了経路へ到達した場合	処理結果を呼出元へ返す

insertTag

処理概要

顧客情報システム共通クラスで「insertTag」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理4：詳細仕様参照）。
2. 関連情報を取得・設定する（処理5：詳細仕様参照）。
3. 「条件6を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：新しいデータを登録する。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

decodeSmallKana

処理概要

顧客情報システム共通クラスで「decodeSmallKana」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 関連情報を取得・設定する（処理14：詳細仕様参照）。
3. 関連情報を取得・設定する（処理15：詳細仕様参照）。
4. 関連情報を取得・設定する（処理16：詳細仕様参照）。
5. 繰返し条件「条件17を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理24：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

zenKanamojiHenkan

処理概要

顧客情報システム共通クラスで「zenKanamojiHenkan」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「ぽ」を設定する。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 繰返し条件「条件15を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

zenKataKanamojiHenkan

処理概要

顧客情報システム共通クラスで「zenKataKanamojiHenkan」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「ポ」を設定する。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 繰返し条件「条件15を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

zenkakumokiHenkan

処理概要

顧客情報システム共通クラスで「zenkakumokiHenkan」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「ン」を設定する。

2. 関連情報を取得・設定する（処理12：詳細仕様参照）。

3. 関連情報を取得・設定する（処理13：詳細仕様参照）。

4. 関連情報を取得・設定する（処理14：詳細仕様参照）。

5. 繰返し条件「条件15を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

decodeSmallzenKana

処理概要

顧客情報システム共通クラスで「decodeSmallzenKana」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「っ」を設定する。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 繰返し条件「条件15を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

decodeSmallkatakana

処理概要

顧客情報システム共通クラスで「decodeSmallkatakana」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「ッ」を設定する。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。
3. 関連情報を取得・設定する（処理13：詳細仕様参照）。
4. 関連情報を取得・設定する（処理14：詳細仕様参照）。
5. 繰返し条件「条件15を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理17：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

alphabetZenHenkan

処理概要

顧客情報システム共通クラスで「alphabetZenHenkan」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 関連情報を取得・設定する（処理14：詳細仕様参照）。
3. 関連情報を取得・設定する（処理15：詳細仕様参照）。
4. 関連情報を取得・設定する（処理16：詳細仕様参照）。
5. 繰返し条件「条件17を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：関連情報を取得・設定する（処理18：詳細仕様参照）。

5.2. 繰返しの各回：繰返し条件「index が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.2.1. 繰返しの各回：関連情報を取得・設定する（処理19：詳細仕様参照）。

5.2.2. 繰返しの各回：「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.2.2.1. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

5.2.2.2. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

5.2.2.3. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

5.2.3. 繰返しの各回：関連情報を取得・設定する（処理24：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

decodeLongMinus

処理概要

顧客情報システム共通クラスで「decodeLongMinus」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理9：詳細仕様参照）。

2. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：指定した条件でデータを照会する。

2-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 繰返し条件「checkString.indexOf("?") が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

4.2. 繰返しの各回：関連情報を取得・設定する（処理13：詳細仕様参照）。

4.3. 繰返しの各回：関連情報を取得・設定する（処理14：詳細仕様参照）。

4.4. 繰返しの各回：関連情報を取得・設定する（処理15：詳細仕様参照）。

4.5. 繰返しの各回：関連情報を取得・設定する（処理16：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editNewLine

処理概要

顧客情報システム共通クラスで「editNewLine」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 繰返し条件「s.indexOf("\r\n") が異なる -1」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理7：詳細仕様参照）。

2.2. 繰返しの各回：関連情報を取得・設定する（処理8：詳細仕様参照）。

2.3. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

2.4. 繰返しの各回：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editKanma

処理概要

顧客情報システム共通クラスで「editKanma」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editKanma

処理概要

顧客情報システム共通クラスで「editKanma」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editKanma

処理概要

顧客情報システム共通クラスで「editKanma」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理27：詳細仕様参照）。
2. 関連情報を取得・設定する（処理28：詳細仕様参照）。
3. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

3.1. 通常処理：関連情報を取得・設定する（処理29：詳細仕様参照）。

3-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

3-例外1.1. 例外発生時：関連情報を取得・設定する（処理30：詳細仕様参照）。

3-例外1.2. 例外発生時：関連情報を取得・設定する（処理31：詳細仕様参照）。

1. 関連情報を取得・設定する（処理32：詳細仕様参照）。
2. 関連情報を取得・設定する（処理33：詳細仕様参照）。
3. 繰返し条件「条件34を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

6.1. 繰返しの各回：関連情報を取得・設定する（処理35：詳細仕様参照）。

6.2. 繰返しの各回：「条件36を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

6.2.1. 条件成立時：関連情報を取得・設定する（処理37：詳細仕様参照）。

6.2-別.1. 条件不成立時：関連情報を取得・設定する（処理38：詳細仕様参照）。

6.3. 繰返しの各回：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

6.3.1. 通常処理：関連情報を取得・設定する（処理39：詳細仕様参照）。

6.3-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

6.3-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 繰返し条件「条件40を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

7.1. 繰返しの各回：「editKingaku.substring(0,1).compareTo("0") が等しい 0」に該当するか確認する。成立時と不成立時の手順を分けて行う。

7.1.1. 条件成立時：関連情報を取得・設定する（処理41：詳細仕様参照）。

7.1-別.1. 条件不成立時：関連情報を取得・設定する（処理42：詳細仕様参照）。

1. 関連情報を取得・設定する（処理43：詳細仕様参照）。
2. 関連情報を取得・設定する（処理44：詳細仕様参照）。
3. 関連情報を取得・設定する（処理45：詳細仕様参照）。
4. 「setPoint が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.1. 条件成立時：関連情報を取得・設定する（処理46：詳細仕様参照）。

1. 繰返し条件「条件47を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

12.1. 繰返しの各回：関連情報を取得・設定する（処理48：詳細仕様参照）。

12.2. 繰返しの各回：関連情報を取得・設定する（処理49：詳細仕様参照）。

12.3. 繰返しの各回：関連情報を取得・設定する（処理50：詳細仕様参照）。

1. 「条件51を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

13.1. 条件成立時：関連情報を取得・設定する（処理52：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件53を満たす（原本条件は補足説明を参照）・条件54を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
この終了経路へ到達した場合	処理結果を呼出元へ返す

editTelphone

処理概要

顧客情報システム共通クラスで「editTelphone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理3：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 「telphone.substring(0,1) が異なる "0"」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：値に応じた分岐の詳細は原本対応を参照する。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない	処理結果を呼出元へ返す
NOT (telphone が等しい 未設定)	処理結果を呼出元へ返す

editTelphone

処理概要

顧客情報システム共通クラスで「editTelphone」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editDate

処理概要

顧客情報システム共通クラスで「editDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「eDate.length() が等しい 8」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1. 条件成立時：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.2. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.3. 条件成立時：新しいデータを登録する。

1.4. 条件成立時：新しいデータを登録する。

1.5. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

1.6. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-別.1. 条件不成立時：「eDate.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1-別.1.1. 条件成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1-別.1.2. 条件成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

1-別.1.3. 条件成立時：新しいデータを登録する。

1-別.1.4. 条件成立時：新しいデータを登録する。

1-別.1.5. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

1-別.1.6. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入

情報	内容
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

結果・例外時の扱い

到達条件	結果
eDate.length() が等しい 8	処理結果を呼出元へ返す
NOT (eDate.length() が等しい 8)・eDate.length() が等しい 6	処理結果を呼出元へ返す
NOT (eDate.length() が等しい 8)・NOT (eDate.length() が等しい 6)	処理結果を呼出元へ返す

edit49Date

処理概要

顧客情報システム共通クラスで「edit49Date」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理2：詳細仕様参照）。
2. 対象データを削除する。
3. 対象データを削除する。
4. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?

情報	内容
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

editTime

処理概要

顧客情報システム共通クラスで「editTime」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「eTime.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1. 条件成立時：関連情報を取得・設定する（処理4：詳細仕様参照）。

1.2. 条件成立時：関連情報を取得・設定する（処理5：詳細仕様参照）。

1.3. 条件成立時：新しいデータを登録する。

1.4. 条件成立時：新しいデータを登録する。

1.5. 条件成立時：関連情報を取得・設定する（処理6：詳細仕様参照）。

1.6. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352

情報	内容
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-1A463E2BC06C

情報	内容
操作ID	SQL-1A463E2BC06C
対象テーブル	年金1_LK
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:661 / insert@657

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, Integer.valueOf(0))

結果・例外時の扱い

到達条件	結果
eTime.length() が等しい 6	処理結果を呼出元へ返す
NOT (eTime.length() が等しい 6)	処理結果を呼出元へ返す

display

処理概要

顧客情報システム共通クラスで「display」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「対象項目がある」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1.1. 条件成立時：「<h2>サーブレットによるサンプルデータベースの検索結果</h2>」を設定する。

1.1.2. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1.1.3. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

1.1.4. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1.5. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

1.1.6. 条件成立時：繰返し条件「条件22を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

1.1.6.1. 繰返しの各回：関連情報を取得・設定する（処理23：詳細仕様参照）。

1.1.7. 条件成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

1.1.8. 条件成立時：繰返し条件「条件25を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

1.1.8.1. 繰返しの各回：関連情報を取得・設定する（処理26：詳細仕様参照）。

1.1.9. 条件成立時：関連情報を取得・設定する（処理27：詳細仕様参照）。

1.1.10. 条件成立時：「条件28を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.10.1. 条件成立時：関連情報を取得・設定する（処理29：詳細仕様参照）。

1.1.10.2. 条件成立時：繰返し条件「条件30を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

1.1.10.3. 条件成立時：関連情報を取得・設定する（処理31：詳細仕様参照）。

1.1.10.4. 条件成立時：「件のデータが見つかりました。」を設定する。

1.1.10-別.1. 条件不成立時：関連情報を取得・設定する（処理32：詳細仕様参照）。

1.1.10-別.2. 条件不成立時：「検索条件に適合するデータは見つかりませんでした。」を設定する。

1-例外1. 例外条件「SQLException ex」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：関連情報を取得・設定する（処理33：詳細仕様参照）。

1-例外1.2. 例外発生時：関連情報を取得・設定する（処理34：詳細仕様参照）。

getSeireki

処理概要

顧客情報システム共通クラスで「getSeireki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理45：詳細仕様参照）。
2. 関連情報を取得・設定する（処理46：詳細仕様参照）。
3. 関連情報を取得・設定する（処理47：詳細仕様参照）。
4. 関連情報を取得・設定する（処理48：詳細仕様参照）。
5. 関連情報を取得・設定する（処理49：詳細仕様参照）。

6. 関連情報を取得・設定する（処理50：詳細仕様参照）。

7. 関連情報を取得・設定する（処理51：詳細仕様参照）。

8. 関連情報を取得・設定する（処理52：詳細仕様参照）。

9. 「wareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.1. 条件成立時：関連情報を取得・設定する（処理53：詳細仕様参照）。

9.2. 条件成立時：関連情報を取得・設定する（処理54：詳細仕様参照）。

9.3. 条件成立時：関連情報を取得・設定する（処理55：詳細仕様参照）。

9.4. 条件成立時：関連情報を取得・設定する（処理56：詳細仕様参照）。

9.5. 条件成立時：「条件57を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5.1. 条件成立時：関連情報を取得・設定する（処理58：詳細仕様参照）。

9.5-別.1. 条件不成立時：「条件59を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理60：詳細仕様参照）。

9.5-別.1-別.1. 条件不成立時：「条件61を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理62：詳細仕様参照）。

9.5-別.1-別.1-別.1. 条件不成立時：「条件63を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理64：詳細仕様参照）。

9.5-別.1-別.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理65：詳細仕様参照）。

9.6. 条件成立時：関連情報を取得・設定する（処理66：詳細仕様参照）。

9.7. 条件成立時：関連情報を取得・設定する（処理67：詳細仕様参照）。

9.8. 条件成立時：関連情報を取得・設定する（処理68：詳細仕様参照）。

9-別.1. 条件不成立時：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

9-別.1.1. 通常処理：関連情報を取得・設定する（処理69：詳細仕様参照）。

9-別.1-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

9-別.1-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

9-別.2. 条件不成立時：「wareki.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2.1. 条件成立時：関連情報を取得・設定する（処理70：詳細仕様参照）。

9-別.2.2. 条件成立時：関連情報を取得・設定する（処理71：詳細仕様参照）。

9-別.2.3. 条件成立時：関連情報を取得・設定する（処理72：詳細仕様参照）。

9-別.2.4. 条件成立時：関連情報を取得・設定する（処理73：詳細仕様参照）。

9-別.2.5. 条件成立時：関連情報を取得・設定する（処理74：詳細仕様参照）。

9-別.2.6. 条件成立時：関連情報を取得・設定する（処理75：詳細仕様参照）。

9-別.2.7. 条件成立時：関連情報を取得・設定する（処理76：詳細仕様参照）。

9-別.2-別.1. 条件不成立時：「wareki.length() が等しい 4」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2-別.1.1. 条件成立時：関連情報を取得・設定する（処理77：詳細仕様参照）。

9-別.2-別.1.2. 条件成立時：関連情報を取得・設定する（処理78：詳細仕様参照）。

9-別.2-別.1.3. 条件成立時：関連情報を取得・設定する（処理79：詳細仕様参照）。

9-別.2-別.1.4. 条件成立時：関連情報を取得・設定する（処理80：詳細仕様参照）。

9-別.2-別.1.5. 条件成立時：関連情報を取得・設定する（処理81：詳細仕様参照）。

9-別.2-別.1.6. 条件成立時：関連情報を取得・設定する（処理82：詳細仕様参照）。

9-別.2-別.1-別.1. 条件不成立時：「wareki.length() が等しい 2」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理83：詳細仕様参照）。

9-別.2-別.1-別.1.2. 条件成立時：関連情報を取得・設定する（処理84：詳細仕様参照）。

9-別.2-別.1-別.1.3. 条件成立時：関連情報を取得・設定する（処理85：詳細仕様参照）。

9-別.2-別.1-別.1.4. 条件成立時：関連情報を取得・設定する（処理86：詳細仕様参照）。

9-別.2-別.1-別.1.5. 条件成立時：関連情報を取得・設定する（処理87：詳細仕様参照）。

9-別.2-別.1-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理88：詳細仕様参照）。
2. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
NOT (wareki.length() が等しい 7)・条件89を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (wareki.length() が等しい 7)・NOT (wareki.length() が等しい 6)・NOT (wareki.length() が等しい 4)・NOT (wareki.length() が等しい 2)	処理結果を呼出元へ返す
check が等しい いいえ	処理結果を呼出元へ返す
NOT (check が等しい いいえ)	処理結果を呼出元へ返す

checkDate

処理概要

顧客情報システム共通クラスで「checkDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkDate

処理概要

顧客情報システム共通クラスで「checkDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「条件5を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。

2. 関連情報を取得・設定する（処理7：詳細仕様参照）。
3. 関連情報を取得・設定する（処理8：詳細仕様参照）。
4. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件9を満たす（原本条件は補足説明を参照）	不成立の結果を返す
条件10を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

checkDate

処理概要

顧客情報システム共通クラスで「checkDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkDate

処理概要

顧客情報システム共通クラスで「checkDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理5：詳細仕様参照）。
2. 「対象項目」を設定する。
3. 関連情報を取得・設定する（処理6：詳細仕様参照）。
4. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

4-例外1. 例外条件「IllegalArgumentException iae」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkWarekiDate

処理概要

顧客情報システム共通クラスで「checkWarekiDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 「wareki.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

3-別.1. 条件不成立時：「wareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3-別.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

3-別.1.2. 条件成立時：「条件14を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3-別.1.2.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

3-別.1.2-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

4-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
NOT (wareki.length() が等しい 6)・wareki.length() が等しい 7・条件19を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (wareki.length() が等しい 6)・NOT (wareki.length() が等しい 7)	処理結果を呼出元へ返す
条件20を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSanissu

処理概要

顧客情報システム共通クラスで「getSanissu」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理11：詳細仕様参照）。
2. 関連情報を取得・設定する（処理12：詳細仕様参照）。

3. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理14：詳細仕様参照）。
2. 関連情報を取得・設定する（処理15：詳細仕様参照）。
3. 関連情報を取得・設定する（処理16：詳細仕様参照）。
4. 関連情報を取得・設定する（処理17：詳細仕様参照）。
5. 関連情報を取得・設定する（処理18：詳細仕様参照）。
6. 関連情報を取得・設定する（処理19：詳細仕様参照）。
7. 関連情報を取得・設定する（処理20：詳細仕様参照）。
8. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
check が等しい いいえ	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・check が等しい いいえ	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・NOT (check が等しい いいえ)	処理結果を呼出元へ返す

getSanissu

処理概要

顧客情報システム共通クラスで「getSanissu」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.2. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.3. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

1-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

2.3. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件13を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件14を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSanissu

処理概要

顧客情報システム共通クラスで「getSanissu」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「startDate.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「endDate.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 関連情報を取得・設定する（処理9：詳細仕様参照）。
4. 関連情報を取得・設定する（処理10：詳細仕様参照）。

5. 関連情報を取得・設定する（処理11：詳細仕様参照）。
6. 関連情報を取得・設定する（処理12：詳細仕様参照）。
7. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
startDate.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (startDate.length() が異なる 8)・endDate.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (startDate.length() が異なる 8)・NOT (endDate.length() が異なる 8)	処理結果を呼出元へ返す

getSanissu

処理概要

顧客情報システム共通クラスで「getSanissu」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 繰返し条件「条件8を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getAge

処理概要

顧客情報システム共通クラスで「getAge」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 繰返し条件「条件8を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理9：詳細仕様参照）。

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 繰返し条件「条件11を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理12：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getAge

処理概要

顧客情報システム共通クラスで「getAge」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「startDate.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「endDate.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

2.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理7：詳細仕様参照）。
2. 関連情報を取得・設定する（処理8：詳細仕様参照）。
3. 関連情報を取得・設定する（処理9：詳細仕様参照）。
4. 関連情報を取得・設定する（処理10：詳細仕様参照）。
5. 関連情報を取得・設定する（処理11：詳細仕様参照）。
6. 関連情報を取得・設定する（処理12：詳細仕様参照）。
7. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
startDate.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (startDate.length() が異なる 8)・endDate.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (startDate.length() が異なる 8)・NOT (endDate.length() が異なる 8)	処理結果を呼出元へ返す

getAge

処理概要

顧客情報システム共通クラスで「getAge」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.2. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.3. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

1-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理10：詳細仕様参照）。

2.2. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

2.3. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
条件13を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件14を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
この終了経路へ到達した場合	処理結果を呼出元へ返す

getAge

処理概要

顧客情報システム共通クラスで「getAge」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理30：詳細仕様参照）。
2. 繰返し条件「条件31を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

2.1. 繰返しの各回：関連情報を取得・設定する（処理32：詳細仕様参照）。

1. 関連情報を取得・設定する（処理33：詳細仕様参照）。
2. 繰返し条件「条件34を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：関連情報を取得・設定する（処理35：詳細仕様参照）。

1. 関連情報を取得・設定する（処理36：詳細仕様参照）。
2. 繰返し条件「条件37を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

6.1. 繰返しの各回：関連情報を取得・設定する（処理38：詳細仕様参照）。

1. 関連情報を取得・設定する（処理39：詳細仕様参照）。
2. 繰返し条件「条件40を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

8.1. 繰返しの各回：関連情報を取得・設定する（処理41：詳細仕様参照）。

1. 関連情報を取得・設定する（処理42：詳細仕様参照）。
2. 繰返し条件「条件43を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

10.1. 繰返しの各回：関連情報を取得・設定する（処理44：詳細仕様参照）。

1. 関連情報を取得・設定する（処理45：詳細仕様参照）。
2. 繰返し条件「条件46を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

12.1. 繰返しの各回：関連情報を取得・設定する（処理47：詳細仕様参照）。

1. 関連情報を取得・設定する（処理48：詳細仕様参照）。
2. 関連情報を取得・設定する（処理49：詳細仕様参照）。
3. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

15.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理50：詳細仕様参照）。
2. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

17.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件51を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

18.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

18-別.1. 条件不成立時：「startYear が等しい endYear」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

18-別.1.1. 条件成立時：「条件52を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

18-別.1.1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

18-別.1.1-別.1. 条件不成立時：「endMonth が等しい startMonth」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

18-別.1.1-別.1.1. 条件成立時：「条件53を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

18-別.1.1-別.1.1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理54：詳細仕様参照）。
2. 「条件55を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

20.1. 条件成立時：関連情報を取得・設定する（処理56：詳細仕様参照）。

20-別.1. 条件不成立時：「endMonth が等しい startMonth」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

20-別.1.1. 条件成立時：「条件57を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

20-別.1.1.1. 条件成立時：関連情報を取得・設定する（処理58：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
check が等しい いいえ	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・check が等しい いいえ	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・NOT (check が等しい いいえ)・条件59を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・NOT (check が等しい いいえ)・条件60を満たす（原本条件は補足説明を参照）・startYear が等しい endYear・条件61を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・NOT (check が等しい いいえ)・条件62を満たす（原本条件は補足説明を参照）・startYear が等しい endYear・条件63を満たす（原本条件は補足説明を参照）・endMonth が等しい startMonth・条件64を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (check が等しい いいえ)・NOT (check が等しい いいえ)・条件65を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

checkEigyobi

処理概要

顧客情報システム共通クラスで「checkEigyobi」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

checkEigyobi

処理概要

顧客情報システム共通クラスで「checkEigyobi」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「seireki.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理8：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件9を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「入力値の制約と変換結果が成立する」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 関連情報を取得・設定する（処理12：詳細仕様参照）。
4. 「対象項目」を設定する。
5. 関連情報を取得・設定する（処理13：詳細仕様参照）。
6. 関連情報を取得・設定する（処理14：詳細仕様参照）。
7. 「youbi が等しい 1、またはyoubi が等しい 7」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.1. 条件成立時：不成立の結果を返す。この経路の後続手順は実行しない。

1. 成立の結果を返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
seireki.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件15を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件16を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件17を満たす（原本条件は補足説明を参照）・入力値の制約と変換結果が成立する	不成立の結果を返す
NOT (seireki.length() が異なる 8)・条件18を満たす（原本条件は補足説明を参照）・条件19を満たす（原本条件は補足説明を参照）・youbi が等しい 1、またはyoubi が等しい 7	不成立の結果を返す
NOT (seireki.length() が異なる 8)・条件20を満たす（原本条件は補足説明を参照）・条件21を満たす（原本条件は補足説明を参照）・NOT (youbi が等しい 1 || youbi が等しい 7)	成立の結果を返す

getSanissuDate

処理概要

顧客情報システム共通クラスで「getSanissuDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSanissuDate

処理概要

顧客情報システム共通クラスで「getSanissuDate」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「seireki.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理19：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件20を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「sanissu が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理21：詳細仕様参照）。
2. 関連情報を取得・設定する（処理22：詳細仕様参照）。
3. 関連情報を取得・設定する（処理23：詳細仕様参照）。
4. 「対象項目」を設定する。
5. 関連情報を取得・設定する（処理24：詳細仕様参照）。
6. 関連情報を取得・設定する（処理25：詳細仕様参照）。
7. 関連情報を取得・設定する（処理26：詳細仕様参照）。
8. 関連情報を取得・設定する（処理27：詳細仕様参照）。
9. 関連情報を取得・設定する（処理28：詳細仕様参照）。
10. 関連情報を取得・設定する（処理29：詳細仕様参照）。
11. 関連情報を取得・設定する（処理30：詳細仕様参照）。
12. 関連情報を取得・設定する（処理31：詳細仕様参照）。
13. 関連情報を取得・設定する（処理32：詳細仕様参照）。
14. 繰返し条件「条件33を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

18.1. 繰返しの各回：関連情報を取得・設定する（処理34：詳細仕様参照）。

    1. 繰返し条件「条件35を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

19.1. 繰返しの各回：関連情報を取得・設定する（処理36：詳細仕様参照）。

    1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
seireki.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件37を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件38を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件39を満たす（原本条件は補足説明を参照）・sanissu が等しい 0	処理結果を呼出元へ返す
NOT (seireki.length() が異なる 8)・条件40を満たす（原本条件は補足説明を参照）・NOT (sanissu が等しい 0)	処理結果を呼出元へ返す

getSaeigyoubi

処理概要

顧客情報システム共通クラスで「getSaeigyoubi」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

    1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getSaeigyoubi

処理概要

顧客情報システム共通クラスで「getSaeigyoubi」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「seireki.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理11：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件12を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「saeigyoubisu が等しい 0」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 関連情報を取得・設定する（処理14：詳細仕様参照）。
3. 関連情報を取得・設定する（処理15：詳細仕様参照）。
4. 「条件16を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

8.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

8.2. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 繰返し条件「条件19を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

9.1. 繰返しの各回：繰返し条件「条件20を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
seireki.length() が異なる 8	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件21を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件22を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (seireki.length() が異なる 8)・条件23を満たす（原本条件は補足説明を参照）・saeigyoubisu が等しい 0	処理結果を呼出元へ返す
NOT (seireki.length() が異なる 8)・条件24を満たす（原本条件は補足説明を参照）・NOT (saeigyoubisu が等しい 0)	処理結果を呼出元へ返す

getSeirekiForWareki

処理概要

顧客情報システム共通クラスで「getSeirekiForWareki」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「対象項目がある、かつwareki.length() が等しい 11」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

1.1.2. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

1.1.3. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

1.1.4. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

1.1.5. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

1.1.6. 条件成立時：「条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1.6.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1.7. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

1.1.8. 条件成立時：「gengo.equals("明治")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8.1. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1.8-別.1. 条件不成立時：「gengo.equals("大正")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1.1. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

1.1.8-別.1-別.1. 条件不成立時：「gengo.equals("昭和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

1.1.8-別.1-別.1-別.1. 条件不成立時：「gengo.equals("平成")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

1.1.8-別.1-別.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

1.1.9. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がある、かつwareki.length() が等しい 11・条件25を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がある、かつwareki.length() が等しい 11・条件26を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (wareki が異なる 未設定 && wareki.length() が等しい 11)	処理結果を呼出元へ返す
条件27を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

convertWareki2

処理概要

顧客情報システム共通クラスで「convertWareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がない、または(seireki.length() が異なる 8)」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理13：詳細仕様参照）。
2. 繰返し条件「条件14を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

4.1. 繰返しの各回：「条件15を満たす（原本条件は補足説明を参照）、かつ条件16を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

4.1.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

1. 「gengoTable2[i][0].equals("不明")」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理18：詳細仕様参照）。

2. 関連情報を取得・設定する（処理19：詳細仕様参照）。

3. 関連情報を取得・設定する（処理20：詳細仕様参照）。

4. 「条件21を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

9.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

1. 「日」を設定する。

2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がない、または(seireki.length() が異なる 8)	処理結果を呼出元へ返す
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・条件23を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・gengoTable2[i][0].equals("不明")	処理結果を呼出元へ返す
NOT (seireki が等しい 未設定 || (seireki.length() が異なる 8))・NOT (gengoTable2[i][0].equals("不明"))	処理結果を呼出元へ返す

convertSimpleWareki2

処理概要

顧客情報システム共通クラスで「convertSimpleWareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：関連情報を取得・設定する（処理9：詳細仕様参照）。

1.2. 通常処理：「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2.1. 条件成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1.2-別.1. 条件不成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

1.2-別.2. 条件不成立時：対象データを削除する。

1.2-別.3. 条件不成立時：対象データを削除する。

1.2-別.4. 条件不成立時：対象データを削除する。

1.2-別.5. 条件不成立時：「sbDate.substring(0,2).equals("令和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2-別.5.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

1.2-別.5-別.1. 条件不成立時：「sbDate.substring(0,2).equals("平成")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2-別.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

1.2-別.5-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("昭和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2-別.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

1.2-別.5-別.1-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("大正")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2-別.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

1.2-別.5-別.1-別.1-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("明治")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.2-別.5-別.1-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

1.2-別.5-別.1-別.1-別.1-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.3. 通常処理：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「Exception e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
NOT (seireki が等しい 未設定)・NOT (sbDate.substring(0,2).equals("令和"))・NOT (sbDate.substring(0,2).equals("平成"))・NOT (sbDate.substring(0,2).equals("昭和"))・NOT (sbDate.substring(0,2).equals("大正"))・NOT (sbDate.substring(0,2).equals("明治"))	処理結果を呼出元へ返す
この終了経路へ到達した場合	処理結果を呼出元へ返す
条件17を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

convertSeireki2

処理概要

顧客情報システム共通クラスで「convertSeireki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「対象項目がある、かつwareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

1.1.2. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1.1.3. 条件成立時：「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1.3.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1.4. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1.5. 条件成立時：「条件21を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.5.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

1.1.5-別.1. 条件不成立時：「条件23を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

1.1.5-別.1-別.1. 条件不成立時：「条件25を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理26：詳細仕様参照）。

1.1.5-別.1-別.1-別.1. 条件不成立時：「条件27を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理28：詳細仕様参照）。

1.1.5-別.1-別.1-別.1-別.1. 条件不成立時：「条件29を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.5-別.1-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理30：詳細仕様参照）。

1.1.5-別.1-別.1-別.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理31：詳細仕様参照）。

1.1.6. 条件成立時：「条件32を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.6.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1.6-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
対象項目がある、かつwareki.length() が等しい 7・条件33を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がある、かつwareki.length() が等しい 7・条件34を満たす（原本条件は補足説明を参照）・条件35を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がある、かつwareki.length() が等しい 7・条件36を満たす（原本条件は補足説明を参照）・条件37を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (wareki が異なる 未設定 && wareki.length() が等しい 7)	処理結果を呼出元へ返す
条件38を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

getSeirekiForWareki2

処理概要

顧客情報システム共通クラスで「getSeirekiForWareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

1.1. 通常処理：「対象項目がある、かつwareki.length() が等しい 11」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

1.1.2. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

1.1.3. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

1.1.4. 条件成立時：関連情報を取得・設定する（処理17：詳細仕様参照）。

1.1.5. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1.1.6. 条件成立時：「条件19を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1.6.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1.7. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1.8. 条件成立時：「gengo.equals("明治")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8.1. 条件成立時：関連情報を取得・設定する（処理21：詳細仕様参照）。

1.1.8-別.1. 条件不成立時：「gengo.equals("大正")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1.1. 条件成立時：関連情報を取得・設定する（処理22：詳細仕様参照）。

1.1.8-別.1-別.1. 条件不成立時：「gengo.equals("昭和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理23：詳細仕様参照）。

1.1.8-別.1-別.1-別.1. 条件不成立時：「gengo.equals("平成")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理24：詳細仕様参照）。

1.1.8-別.1-別.1-別.1-別.1. 条件不成立時：「gengo.equals("令和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.8-別.1-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理25：詳細仕様参照）。

1.1.8-別.1-別.1-別.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理26：詳細仕様参照）。

1.1.9. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-例外1. 例外条件「NumberFormatException e」に該当した場合に次の手順を行う。型名は原本の表記。

1-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がある、かつwareki.length() が等しい 11・条件27を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
対象項目がある、かつwareki.length() が等しい 11・条件28を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (wareki が異なる 未設定 && wareki.length() が等しい 11)	処理結果を呼出元へ返す
条件29を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す

getSimpleGengoWareki2

処理概要

顧客情報システム共通クラスで「getSimpleGengoWareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理9：詳細仕様参照）。
2. 「対象項目がない」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

2-別.1. 条件不成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

2-別.2. 条件不成立時：対象データを削除する。

2-別.3. 条件不成立時：対象データを削除する。

2-別.4. 条件不成立時：対象データを削除する。

2-別.5. 条件不成立時：「sbDate.substring(0,2).equals("令和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

2-別.5-別.1. 条件不成立時：「sbDate.substring(0,2).equals("平成")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

2-別.5-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("昭和")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理14：詳細仕様参照）。

2-別.5-別.1-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("大正")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

2-別.5-別.1-別.1-別.1-別.1. 条件不成立時：「sbDate.substring(0,2).equals("明治")」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2-別.5-別.1-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理16：詳細仕様参照）。

2-別.5-別.1-別.1-別.1-別.1-別.1. 条件不成立時：対象データを削除する。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK

情報	内容
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

get49Wareki2

処理概要

顧客情報システム共通クラスで「get49Wareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理6：詳細仕様参照）。
2. 「eDate.length() が等しい 10」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1. 条件成立時：「条件7を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

2.1.1. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

2.1-別.1. 条件不成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

2-別.1. 条件不成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除

情報	内容
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())

設定先のDB項目	バインド順	値の設定・変換（原本）
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
この終了経路へ到達した場合	処理結果を呼出元へ返す

getWareki2

処理概要

顧客情報システム共通クラスで「getWareki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「seireki.length() が異なる 8」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

1.1. 条件成立時：「seireki.length() が異なる 10」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1.1-別.1. 条件不成立時：「条件17を満たす（原本条件は補足説明を参照）、かつ条件18を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理19：詳細仕様参照）。

1.1-別.1.2. 条件成立時：対象データを削除する。

1.1-別.1.3. 条件成立時：対象データを削除する。

1.1-別.1.4. 条件成立時：関連情報を取得・設定する（処理20：詳細仕様参照）。

1.1-別.1-別.1. 条件不成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

2.1. 通常処理：関連情報を取得・設定する（処理21：詳細仕様参照）。

2-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

2-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 「条件22を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

3.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理23：詳細仕様参照）。
2. 繰返し条件「条件24を満たす（原本条件は補足説明を参照）」に従い、次の手順を繰り返す。条件を満たさなくなった場合は繰返しの後へ進む。

5.1. 繰返しの各回：「条件25を満たす（原本条件は補足説明を参照）、かつ条件26を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1.1. 条件成立時：関連情報を取得・設定する（処理27：詳細仕様参照）。

1. 「gengoTable2[i][0].equals("不明")」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

6.1. 条件成立時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理28：詳細仕様参照）。
2. 関連情報を取得・設定する（処理29：詳細仕様参照）。
3. 関連情報を取得・設定する（処理30：詳細仕様参照）。
4. 「条件31を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

10.1. 条件成立時：関連情報を取得・設定する（処理32：詳細仕様参照）。

1. 「日」を設定する。
2. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

DB操作の詳細

DB操作 SQL-F9D3BB2927F4

情報	内容
操作ID	SQL-F9D3BB2927F4
対象テーブル	年金1_LK
操作	削除
処理説明	対象テーブルの条件に一致する行を削除する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：ふりがな = ? AND 電話番号 = ?
確定・取消	当該メソッド内で自動コミット設定・コミット・ロールバックを行う。例外時の処理順は詳細仕様の分岐に従う。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:334 / delete@315

DB操作 SQL-F5C825F11352

情報	内容
操作ID	SQL-F5C825F11352
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。

情報	内容
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：種類ｺｰﾄﾞ = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:609 / resolvePensionTypeCode@596

DB操作 SQL-EC6FD30BE7CE

情報	内容
操作ID	SQL-EC6FD30BE7CE
対象テーブル	年金1_LK
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B8CFB512D81A

情報	内容
操作ID	SQL-B8CFB512D81A
対象テーブル	店名テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-B3BD08EC9265

情報	内容
操作ID	SQL-B3BD08EC9265
対象テーブル	性別テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?

情報	内容
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-47BAF2E0D129

情報	内容
操作ID	SQL-47BAF2E0D129
対象テーブル	裁定請求テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-C66BA4DBA955

情報	内容
操作ID	SQL-C66BA4DBA955
対象テーブル	受給有無テーブル

情報	内容
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-E2E89DE2FC84

情報	内容
操作ID	SQL-E2E89DE2FC84
対象テーブル	年金種類テーブル
操作	参照
処理説明	対象テーブルの対象データを読み取る。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	当該SQL式の列指定・実行時引数を使用する。組立式と原本のパラメーター設定を参照。
対象条件	抽出・更新対象条件：R.ふりがな = ? AND R.電話番号 = ?
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:645 / findByKey@640

DB操作 SQL-D7ADE1A915C2

情報	内容
操作ID	SQL-D7ADE1A915C2
対象テーブル	年金__削除明細
操作	挿入
処理説明	対象テーブルの新しい行を登録する。設定項目・対象条件・確定取消は以下の定義に従う。
設定項目	店番 ← バインド引数1（PreparedStatementまたは呼出側の設定順序）／顧客番号 ← バインド引数2（PreparedStatementまたは呼出側の設定順序）／担当者コード ← バインド引数3（PreparedStatementまたは呼出側の設定順序）／顧客氏名 ← バインド引数4（PreparedStatementまたは呼出側の設定順序）／ふりがな ← バインド引数5（PreparedStatementまたは呼出側の設定順序）／性別 ← バインド引数6（PreparedStatementまたは呼出側の設定順序）／住所 ← バインド引数7（PreparedStatementまたは呼出側の設定順序）／電話番号 ← バインド引数8（PreparedStatementまたは呼出側の設定順序）／生年月日 ← バインド引数9（PreparedStatementまたは呼出側の設定順序）／種類：国民年金 ← バインド引数10（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金 ← バインド引数11（PreparedStatementまたは呼出側の設定順序）／種類：共済年金 ← バインド引数12（PreparedStatementまたは呼出側の設定順序）／種類：厚生年金基金 ← バインド引数13（PreparedStatementまたは呼出側の設定順序）／種類：国民年金基金 ← バインド引数14（PreparedStatementまたは呼出側の設定順序）／年金種類 ← バインド引数15（PreparedStatementまたは呼出側の設定順序）／裁定請求 ← バインド引数16（PreparedStatementまたは呼出側の設定順序）／請求予定日 ← バインド引数17（PreparedStatementまたは呼出側の設定順序）／予定年齢 ← バインド引数18（PreparedStatementまたは呼出側の設定順序）／配偶者名 ← バインド引数19（PreparedStatementまたは呼出側の設定順序）／配偶者生年月日 ← バインド引数20（PreparedStatementまたは呼出側の設定順序）／受給有無 ← バインド引数21（PreparedStatementまたは呼出側の設定順序）／予約徴求日 ← バインド引数22（PreparedStatementまたは呼出側の設定順序）／備考 ← バインド引数23（PreparedStatementまたは呼出側の設定順序）／データ更新日 ← バインド引数24（PreparedStatementまたは呼出側の設定順序）／更新時間 ← バインド引数25（PreparedStatementまたは呼出側の設定順序）／更新者 ← バインド引数26（PreparedStatementまたは呼出側の設定順序）／６ヶ月先確認 ← バインド引数27（PreparedStatementまたは呼出側の設定順序）／変更FLG ← バインド引数28（PreparedStatementまたは呼出側の設定順序）

情報	内容
対象条件	新しい行を挿入する。重複や採番の扱いは呼出元および列制約に従う。
確定・取消	呼出側または接続の設定に従う。当該メソッド単独でのコミット・ロールバック指定はない。 同じ原本に明示トランザクションの呼出元があり、経路別の確定・取消範囲は関連処理で確認する。
原本	nenkin/WEB-INF/src/nenkin/service/PensionRepository.java:694 / insertDeleted@690

設定先のDB項目	バインド順	値の設定・変換（原本）
店番	1	setInteger(statement, 1, value.getBranchNo())
顧客番号	2	setInteger(statement, 2, value.getCustomerNo())
担当者コード	3	setInteger(statement, 3, value.getStaffCode())
顧客氏名	4	value.getCustomerName()
ふりがな	5	value.getKanaName()
性別	6	setInteger(statement, 6, resolveCode(connection, GENDER_TABLE, "性別ｺｰﾄﾞ", "性別", value.getGender()))
住所	7	value.getAddress()
電話番号	8	setPhone(statement, 8, value.getPhoneNumber())
生年月日	9	setDate(statement, 9, value.getBirthDate())
種類：国民年金	10	value.isNationalPension() ? 1 : 0
種類：厚生年金	11	value.isEmployeePension() ? 1 : 0
種類：共済年金	12	value.isMutualAidPension() ? 1 : 0
種類：厚生年金基金	13	value.isEmployeePensionFund() ? 1 : 0

設定先のDB項目	バインド順	値の設定・変換（原本）
種類：国民年金基金	14	value.isNationalPensionFund() ? 1 : 0
年金種類	15	setInteger(statement, 15, resolvePensionTypeCode(connection, value))
裁定請求	16	setInteger(statement, 16, resolveCode(connection, CLAIM_TABLE, "裁請ｺｰﾄﾞ", "裁定請求", value.getClaimStatus()))
請求予定日	17	setDate(statement, 17, value.getClaimScheduledDate())
予定年齢	18	setInteger(statement, 18, value.getPlannedAge())
配偶者名	19	value.getSpouseName()
配偶者生年月日	20	setDate(statement, 20, value.getSpouseBirthDate())
受給有無	21	setInteger(statement, 21, resolveCode(connection, BENEFIT_TABLE, "受給ｺｰﾄﾞ", "受給有無", value.getHasBenefit()))
予約徴求日	22	setDate(statement, 22, value.getReservationRequestDate())
備考	23	value.getNotes()
データ更新日	24	setDate(statement, 24, value.getDataUpdateDate())
更新時間	25	setTime(statement, 25, updateTime)
更新者	26	setInteger(statement, 26, updatedBy)
６ヶ月先確認	27	value.isSixMonthCheck() ? 1 : 0
変更FLG	28	setInteger(statement, 28, null)

結果・例外時の扱い

到達条件	結果
seireki.length() が異なる 8・seireki.length() が異なる 10	処理の異常を通知し、この経路を終了する
seireki.length() が異なる 8・NOT (seireki.length() が異なる 10)・条件33を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件34を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件35を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
条件36を満たす（原本条件は補足説明を参照）・gengoTable2[i][0].equals("不明")	処理の異常を通知し、この経路を終了する
条件37を満たす（原本条件は補足説明を参照）・NOT (gengoTable2[i][0].equals("不明"))	処理結果を呼出元へ返す

getSeireki2

処理概要

顧客情報システム共通クラスで「getSeireki2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理47：詳細仕様参照）。
2. 関連情報を取得・設定する（処理48：詳細仕様参照）。
3. 関連情報を取得・設定する（処理49：詳細仕様参照）。
4. 関連情報を取得・設定する（処理50：詳細仕様参照）。
5. 関連情報を取得・設定する（処理51：詳細仕様参照）。

6. 関連情報を取得・設定する（処理52：詳細仕様参照）。

7. 関連情報を取得・設定する（処理53：詳細仕様参照）。

8. 関連情報を取得・設定する（処理54：詳細仕様参照）。

9. 「wareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.1. 条件成立時：関連情報を取得・設定する（処理55：詳細仕様参照）。

9.2. 条件成立時：関連情報を取得・設定する（処理56：詳細仕様参照）。

9.3. 条件成立時：関連情報を取得・設定する（処理57：詳細仕様参照）。

9.4. 条件成立時：関連情報を取得・設定する（処理58：詳細仕様参照）。

9.5. 条件成立時：「条件59を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5.1. 条件成立時：関連情報を取得・設定する（処理60：詳細仕様参照）。

9.5-別.1. 条件不成立時：「条件61を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1.1. 条件成立時：関連情報を取得・設定する（処理62：詳細仕様参照）。

9.5-別.1-別.1. 条件不成立時：「条件63を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理64：詳細仕様参照）。

9.5-別.1-別.1-別.1. 条件不成立時：「条件65を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理66：詳細仕様参照）。

9.5-別.1-別.1-別.1-別.1. 条件不成立時：「条件67を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9.5-別.1-別.1-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理68：詳細仕様参照）。

9.5-別.1-別.1-別.1-別.1-別.1. 条件不成立時：関連情報を取得・設定する（処理69：詳細仕様参照）。

9.6. 条件成立時：関連情報を取得・設定する（処理70：詳細仕様参照）。

9.7. 条件成立時：関連情報を取得・設定する（処理71：詳細仕様参照）。

9.8. 条件成立時：関連情報を取得・設定する（処理72：詳細仕様参照）。

9-別.1. 条件不成立時：通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

9-別.1.1. 通常処理：関連情報を取得・設定する（処理73：詳細仕様参照）。

9-別.1-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

9-別.1-例外1.1. 例外発生時：処理の異常を通知し、この経路を終了する。この経路の後続手順は実行しない。

9-別.2. 条件不成立時：「wareki.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2.1. 条件成立時：関連情報を取得・設定する（処理74：詳細仕様参照）。

9-別.2.2. 条件成立時：関連情報を取得・設定する（処理75：詳細仕様参照）。

9-別.2.3. 条件成立時：関連情報を取得・設定する（処理76：詳細仕様参照）。

9-別.2.4. 条件成立時：関連情報を取得・設定する（処理77：詳細仕様参照）。

9-別.2.5. 条件成立時：関連情報を取得・設定する（処理78：詳細仕様参照）。

9-別.2.6. 条件成立時：関連情報を取得・設定する（処理79：詳細仕様参照）。

9-別.2.7. 条件成立時：関連情報を取得・設定する（処理80：詳細仕様参照）。

9-別.2-別.1. 条件不成立時：「wareki.length() が等しい 4」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2-別.1.1. 条件成立時：関連情報を取得・設定する（処理81：詳細仕様参照）。

9-別.2-別.1.2. 条件成立時：関連情報を取得・設定する（処理82：詳細仕様参照）。

9-別.2-別.1.3. 条件成立時：関連情報を取得・設定する（処理83：詳細仕様参照）。

9-別.2-別.1.4. 条件成立時：関連情報を取得・設定する（処理84：詳細仕様参照）。

9-別.2-別.1.5. 条件成立時：関連情報を取得・設定する（処理85：詳細仕様参照）。

9-別.2-別.1.6. 条件成立時：関連情報を取得・設定する（処理86：詳細仕様参照）。

9-別.2-別.1-別.1. 条件不成立時：「wareki.length() が等しい 2」に該当するか確認する。成立時と不成立時の手順を分けて行う。

9-別.2-別.1-別.1.1. 条件成立時：関連情報を取得・設定する（処理87：詳細仕様参照）。

9-別.2-別.1-別.1.2. 条件成立時：関連情報を取得・設定する（処理88：詳細仕様参照）。

9-別.2-別.1-別.1.3. 条件成立時：関連情報を取得・設定する（処理89：詳細仕様参照）。

9-別.2-別.1-別.1.4. 条件成立時：関連情報を取得・設定する（処理90：詳細仕様参照）。

9-別.2-別.1-別.1.5. 条件成立時：関連情報を取得・設定する（処理91：詳細仕様参照）。

9-別.2-別.1-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 関連情報を取得・設定する（処理92：詳細仕様参照）。
2. 「check が等しい いいえ」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

11.1. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
NOT (wareki.length() が等しい 7)・条件93を満たす（原本条件は補足説明を参照）	処理の異常を通知し、この経路を終了する
NOT (wareki.length() が等しい 7)・NOT (wareki.length() が等しい 6)・NOT (wareki.length() が等しい 4)・NOT (wareki.length() が等しい 2)	処理結果を呼出元へ返す
check が等しい いいえ	処理結果を呼出元へ返す

到達条件	結果
NOT (check が等しい いいえ)	処理結果を呼出元へ返す

checkWarekiDate2

処理概要

顧客情報システム共通クラスで「checkWarekiDate2」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 関連情報を取得・設定する（処理10：詳細仕様参照）。
2. 関連情報を取得・設定する（処理11：詳細仕様参照）。
3. 「wareki.length() が等しい 6」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3.1. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

3-別.1. 条件不成立時：「wareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3-別.1.1. 条件成立時：関連情報を取得・設定する（処理13：詳細仕様参照）。

3-別.1.2. 条件成立時：「条件14を満たす（原本条件は補足説明を参照）」に該当するか確認する。成立時と不成立時の手順を分けて行う。

3-別.1.2.1. 条件成立時：関連情報を取得・設定する（処理15：詳細仕様参照）。

3-別.1.2-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

3-別.1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 通常処理と例外時の処理を区別して実行する。例外処理は対応する例外が発生した場合に限る。

4.1. 通常処理：関連情報を取得・設定する（処理16：詳細仕様参照）。

4-例外1. 例外条件「NumberFormatException nfe」に該当した場合に次の手順を行う。型名は原本の表記。

4-例外1.1. 例外発生時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1. 「条件17を満たす（原本条件は補足説明を参照）」に該当するか確認する。該当する場合に限り、次の分岐内手順を行う。該当しない場合は分岐の後へ進む。

5.1. 条件成立時：関連情報を取得・設定する（処理18：詳細仕様参照）。

1. 処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
NOT (wareki.length() が等しい 6)・wareki.length() が等しい 7・条件19を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
NOT (wareki.length() が等しい 6)・NOT (wareki.length() が等しい 7)	処理結果を呼出元へ返す
条件20を満たす（原本条件は補足説明を参照）	処理結果を呼出元へ返す
この終了経路へ到達した場合	処理結果を呼出元へ返す

editSimpleWarekiDot

処理概要

顧客情報システム共通クラスで「editSimpleWarekiDot」の要求を受け付ける。入力・確認・データ操作・応答の順と、条件が成立しない場合の処理を以下に示す。

処理手順

1. 「対象項目がある、かつwareki.length() が等しい 7」に該当するか確認する。成立時と不成立時の手順を分けて行う。

1.1. 条件成立時：関連情報を取得・設定する（処理7：詳細仕様参照）。

1.2. 条件成立時：関連情報を取得・設定する（処理8：詳細仕様参照）。

1.3. 条件成立時：関連情報を取得・設定する（処理9：詳細仕様参照）。

1.4. 条件成立時：関連情報を取得・設定する（処理10：詳細仕様参照）。

1.5. 条件成立時：関連情報を取得・設定する（処理11：詳細仕様参照）。

1.6. 条件成立時：関連情報を取得・設定する（処理12：詳細仕様参照）。

1.7. 条件成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

1-別.1. 条件不成立時：処理結果を呼出元へ返す。この経路の後続手順は実行しない。

結果・例外時の扱い

到達条件	結果
対象項目がある、かつwareki.length() が等しい 7	処理結果を呼出元へ返す
NOT (wareki が異なる 未設定 && wareki.length() が等しい 7)	処理結果を呼出元へ返す

DB関連処理

補足説明

適用条件

各仮定のQA番号を確認する。静的に確認できた操作・条件を適用する。

処理・条件の原本対応

文書内の条件番号・処理番号は次の原本表記に対応する。完全な業務用語への置換を行っていない箇所はQA・reportで確認する。

イベント	参照番号	原本条件・処理
CifComon	処理1	gsSFilePath = sSFilePath
CifComon	処理2	gsSFilePath = sSFilePath
getWareki	処理1	wareki = wareki + "0"
getWareki	条件2	year < 10
getWareki	処理3	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable[i][1].substring(0,4)) + 1
getWareki	処理4	wareki = gengoTable[i][0]
getWareki	処理5	String wareki = ""
getWareki	条件6	; i < gengoTable.length;i++
getWareki	処理7	break

イベント	参照番号	原本条件・処理
getWareki	条件8	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable[i][1])
getWareki	条件9	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable[i][2])
getWareki	処理10	int i = 0
getWareki	条件11	!checkDate(seireki)
getWareki	処理12	int wk = Integer.parseInt(seireki)
getWareki	処理13	seireki=sbDate.toString()
getWareki	処理14	StringBuffer sbDate=new StringBuffer(seireki)
getWareki	条件15	seireki.substring(4,5).equals("-")
getWareki	条件16	seireki.substring(7,8).equals("-")
getWareki	条件17	seireki.substring(4,5).equals("-")
getWareki	条件18	seireki.substring(7,8).equals("-")
getWareki	処理19	StringBuffer sbDate=new StringBuffer(seireki)
getWareki	処理20	seireki=sbDate.toString()
getWareki	処理21	int wk = Integer.parseInt(seireki)
getWareki	条件22	!checkDate(seireki)
getWareki	処理23	int i = 0
getWareki	条件24	; i < gengoTable.length;i++
getWareki	条件25	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable[i][1])
getWareki	条件26	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable[i][2])

イベント	参照番号	原本条件・処理
getWareki	処理27	break
getWareki	処理28	String wareki = ""
getWareki	処理29	wareki = gengoTable[i][0]
getWareki	処理30	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable[i][1].substring(0,4)) + 1
getWareki	条件31	year < 10
getWareki	処理32	wareki = wareki + "0"
getWareki	条件33	NOT (seireki.substring(4,5).equals("-")&&seireki.substring(7,8).equals("-"))
getWareki	条件34	NumberFormatException nfe
getWareki	条件35	!checkDate(seireki)
getWareki	条件36	NOT (!checkDate(seireki))
getWareki	条件37	NOT (!checkDate(seireki))
getWarekiHenkan	処理1	wareki = wareki + "0"
getWarekiHenkan	条件2	year < 10
getWarekiHenkan	処理3	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable[i][1].substring(0,4)) + 1
getWarekiHenkan	処理4	wareki = gengoTable[i][0]
getWarekiHenkan	処理5	String wareki = ""
getWarekiHenkan	条件6	; i < gengoTable.length;i++

イベント	参照番号	原本条件・処理
getWarekiHenkan	処理7	break
getWarekiHenkan	条件8	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable[i][1])
getWarekiHenkan	条件9	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable[i][2])
getWarekiHenkan	処理10	int i = 0
getWarekiHenkan	処理11	int wk = Integer.parseInt(seireki)
getWarekiHenkan	処理12	int wk = Integer.parseInt(seireki)
getWarekiHenkan	処理13	int i = 0
getWarekiHenkan	条件14	; i < gengoTable.length;i++
getWarekiHenkan	条件15	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable[i][1])
getWarekiHenkan	条件16	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable[i][2])
getWarekiHenkan	処理17	break
getWarekiHenkan	処理18	String wareki = ""
getWarekiHenkan	処理19	wareki = gengoTable[i][0]
getWarekiHenkan	処理20	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable[i][1].substring(0,4)) + 1
getWarekiHenkan	条件21	year < 10
getWarekiHenkan	処理22	wareki = wareki + "0"
getWarekiHenkan	条件23	NumberFormatException nfe
get49Wareki	処理1	sbDate=new StringBuffer(getWareki(edit49Date(eDate)))
get49Wareki	処理2	sbDate=new StringBuffer("dateError in CifComn")

イベント	参照番号	原本条件・処理
get49Wareki	条件3	eDate.substring(4,5).equals("-")&eDate.substring(7,8).equals("-")
get49Wareki	処理4	sbDate=new StringBuffer(getWareki(eDate))
get49Wareki	処理5	StringBuffer sbDate
get49Wareki	処理6	StringBuffer sbDate
get49Wareki	条件7	eDate.substring(4,5).equals("-")&eDate.substring(7,8).equals("-")
get49Wareki	処理8	sbDate=new StringBuffer(getWareki(edit49Date(eDate)))
get49Wareki	処理9	sbDate=new StringBuffer("dateError in CifComn")
get49Wareki	処理10	sbDate=new StringBuffer(getWareki(eDate))
getSimpleWareki	処理1	StringBuffer sbDate=new StringBuffer(get49Wareki(eDate))
getSimpleWareki	処理2	StringBuffer sbDate=new StringBuffer(get49Wareki(eDate))
getSimpleGengoWareki	処理1	sbDate=new StringBuffer("")
getSimpleGengoWareki	処理2	sbDate.replace(0,2,"H")
getSimpleGengoWareki	処理3	sbDate.replace(0,2,"S")
getSimpleGengoWareki	処理4	sbDate.replace(0,2,"T")
getSimpleGengoWareki	処理5	sbDate.replace(0,2,"M")
getSimpleGengoWareki	処理6	sbDate=new StringBuffer(get49Wareki(eDate))
getSimpleGengoWareki	処理7	StringBuffer sbDate
getSimpleGengoWareki	処理8	StringBuffer sbDate
getSimpleGengoWareki	処理9	sbDate=new StringBuffer("")

イベント	参照番号	原本条件・処理
getSimpleGengoWareki	処理10	sbDate=new StringBuffer(get49Wareki(eDate))
getSimpleGengoWareki	処理11	sbDate.replace(0,2,"H")
getSimpleGengoWareki	処理12	sbDate.replace(0,2,"S")
getSimpleGengoWareki	処理13	sbDate.replace(0,2,"T")
getSimpleGengoWareki	処理14	sbDate.replace(0,2,"M")
getLastMonthDate	処理1	lastDate+=lastDay.toString()
getLastMonthDate	処理2	String lastDate=inDate.substring(0,6)
getLastMonthDate	処理3	Integer lastDay=new Integer(cal.getActualMaximum(Calendar.DAY_OF_MONTH))
getLastMonthDate	処理4	cal.set(yyyy,mm-1,dd)
getLastMonthDate	処理5	int dd=Integer.parseInt(inDate.substring(6))
getLastMonthDate	処理6	int mm=Integer.parseInt(inDate.substring(4,6))
getLastMonthDate	処理7	int yyyy=Integer.parseInt(inDate.substring(0,4))
getLastMonthDate	処理8	Calendar cal=new GregorianCalendar(tz)
getLastMonthDate	処理9	TimeZone tz=TimeZone.getTimeZone("JST")
getLastMonthDate	処理10	TimeZone tz=TimeZone.getTimeZone("JST")
getLastMonthDate	処理11	Calendar cal=new GregorianCalendar(tz)
getLastMonthDate	処理12	int yyyy=Integer.parseInt(inDate.substring(0,4))
getLastMonthDate	処理13	int mm=Integer.parseInt(inDate.substring(4,6))
getLastMonthDate	処理14	int dd=Integer.parseInt(inDate.substring(6))

イベント	参照番号	原本条件・処理
getLastMonthDate	処理15	cal.set(yyyy,mm-1,dd)
getLastMonthDate	処理16	Integer lastDay=new Integer(cal.getActualMaximum(Calendar.DAY_OF_MONTH))
getLastMonthDate	処理17	String lastDate=inDate.substring(0,6)
getLastMonthDate	処理18	lastDate+=lastDay.toString()
CifComonRead	処理1	sBuffer = sBuffer + sWk
CifComonRead	処理2	String sWk = new String(cBuf, 0, iLen)
CifComonRead	処理3	int iLen = 0
CifComonRead	処理4	char[] cBuf = new char[4096]
CifComonRead	処理5	cISR = new InputStreamReader(cFIS, "Windows-31J")
CifComonRead	処理6	cFIS = new FileInputStream(sFilePath)
CifComonRead	処理7	InputStreamReader cISR = null
CifComonRead	処理8	FileInputStream cFIS
CifComonRead	処理9	String sBuffer = ""
CifComonRead	処理10	String sBuffer = ""
CifComonRead	処理11	FileInputStream cFIS
CifComonRead	処理12	InputStreamReader cISR = null
CifComonRead	処理13	cFIS = new FileInputStream(sFilePath)
CifComonRead	処理14	cISR = new InputStreamReader(cFIS, "Windows-31J")
CifComonRead	処理15	char[] cBuf = new char[4096]

イベント	参照番号	原本条件・処理
CifComonRead	処理16	int iLen = 0
CifComonRead	処理17	String sWk = new String(cBuf, 0, iLen)
CifComonRead	処理18	sBuffer = sBuffer + sWk
CifComonHTMLRead	処理1	sReturn = sbScr.toString()
CifComonHTMLRead	条件2	iHit > -1
CifComonHTMLRead	処理3	StringBuffer sbScr = new StringBuffer(sBuffer)
CifComonHTMLRead	処理4	int iHit = sBuffer.indexOf("SERVLETLINE")
CifComonHTMLRead	処理5	sBuffer = CifComonRead(sFilePath)
CifComonHTMLRead	処理6	sBuffer = "<!--INPUT FILE:" + sFilePath + "-->\n"
CifComonHTMLRead	処理7	String sReturn = ""
CifComonHTMLRead	処理8	String sBuffer = ""
CifComonHTMLRead	処理9	String sFilePath = gsSFilePath + "\" + htmlFileName
CifComonHTMLRead	処理10	String sFilePath = gsSFilePath + "\" + htmlFileName
CifComonHTMLRead	処理11	String sBuffer = ""
CifComonHTMLRead	処理12	String sReturn = ""
CifComonHTMLRead	処理13	sBuffer = "<!--INPUT FILE:" + sFilePath + "-->\n"
CifComonHTMLRead	処理14	sBuffer = CifComonRead(sFilePath)
CifComonHTMLRead	処理15	int iHit = sBuffer.indexOf("SERVLETLINE")
CifComonHTMLRead	処理16	StringBuffer sbScr = new StringBuffer(sBuffer)

イベント	参照番号	原本条件・処理
CifComonHTMLRead	条件17	iHit > -1
CifComonHTMLRead	処理18	sReturn = sbScr.toString()
CifComonHTMLRead	条件19	Exception e
insertTag	条件1	i>-1
insertTag	処理2	StringBuffer sb = new StringBuffer(baseString)
insertTag	処理3	int i=baseString.indexOf(tag)
insertTag	処理4	int i=baseString.indexOf(tag)
insertTag	処理5	StringBuffer sb = new StringBuffer(baseString)
insertTag	条件6	i>-1
decodeSmallKana	条件1	;count<smallKana[0].length;count++
decodeSmallKana	処理2	s=s+" in="+index+" co="+count
decodeSmallKana	処理3	checkString=checkStringBuffer.toString()
decodeSmallKana	処理4	checkStringBuffer.replace(index,index+1,smallKana[1][count])
decodeSmallKana	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallKana	条件6	index>-1
decodeSmallKana	処理7	index=checkString.indexOf(smallKana[0][count])
decodeSmallKana	処理8	int index=0
decodeSmallKana	処理9	int count=0
decodeSmallKana	処理10	checkString=argCheckData

イベント	参照番号	原本条件・処理
decodeSmallKana	処理11	String checkString=""
decodeSmallKana	処理12	String[][] smallKana = { {"ァ","ィ","ゥ","ェ","ォ","ャ","ュ","ョ","ッ","ｰ"}, {"ｱ","ｲ","ｳ","ｴ","ｵ","ﾔ","ﾕ","ﾖ","ﾂ","-"} }
decodeSmallKana	処理13	String[][] smallKana = { {"ァ","ィ","ゥ","ェ","ォ","ャ","ュ","ョ","ッ","ｰ"}, {"ｱ","ｲ","ｳ","ｴ","ｵ","ﾔ","ﾕ","ﾖ","ﾂ","-"} }
decodeSmallKana	処理14	String checkString=""
decodeSmallKana	処理15	checkString=argCheckData
decodeSmallKana	処理16	int count=0
decodeSmallKana	条件17	;count<smallKana[0].length;count++
decodeSmallKana	処理18	int index=0
decodeSmallKana	処理19	index=checkString.indexOf(smallKana[0][count])
decodeSmallKana	条件20	index>-1
decodeSmallKana	処理21	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallKana	処理22	checkStringBuffer.replace(index,index+1,smallKana[1][count])
decodeSmallKana	処理23	checkString=checkStringBuffer.toString()
decodeSmallKana	処理24	s=s+" in="+index+" co="+count
zenKanamojiHenkan	条件1	;count<Kananmoji[0].length;count++
zenKanamojiHenkan	処理2	s=s+" in="+index+" co="+count
zenKanamojiHenkan	処理3	checkString=checkStringBuffer.toString()
zenKanamojiHenkan	処理4	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])

イベント	参照番号	原本条件・処理
zenKanamojiHenkan	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenKanamojiHenkan	条件6	index>-1
zenKanamojiHenkan	処理7	index=checkString.indexOf(Kananmoji[0][count])
zenKanamojiHenkan	処理8	int index=0
zenKanamojiHenkan	処理9	int count=0
zenKanamojiHenkan	処理10	checkString=argCheckData
zenKanamojiHenkan	処理11	String checkString=""
zenKanamojiHenkan	処理12	String checkString=""
zenKanamojiHenkan	処理13	checkString=argCheckData
zenKanamojiHenkan	処理14	int count=0
zenKanamojiHenkan	条件15	;count<Kananmoji[0].length;count++
zenKanamojiHenkan	処理16	int index=0
zenKanamojiHenkan	処理17	index=checkString.indexOf(Kananmoji[0][count])
zenKanamojiHenkan	条件18	index>-1
zenKanamojiHenkan	処理19	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenKanamojiHenkan	処理20	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])
zenKanamojiHenkan	処理21	checkString=checkStringBuffer.toString()
zenKanamojiHenkan	処理22	s=s+" in="+index+" co="+count
zenKataKanamojiHenkan	条件1	;count<Kananmoji[0].length;count++

イベント	参照番号	原本条件・処理
zenKataKanamojiHenkan	処理2	s=s+" in="+index+" co="+count
zenKataKanamojiHenkan	処理3	checkString=checkStringBuffer.toString()
zenKataKanamojiHenkan	処理4	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])
zenKataKanamojiHenkan	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenKataKanamojiHenkan	条件6	index>-1
zenKataKanamojiHenkan	処理7	index=checkString.indexOf(Kananmoji[0][count])
zenKataKanamojiHenkan	処理8	int index=0
zenKataKanamojiHenkan	処理9	int count=0
zenKataKanamojiHenkan	処理10	checkString=argCheckData
zenKataKanamojiHenkan	処理11	String checkString=""
zenKataKanamojiHenkan	処理12	String checkString=""
zenKataKanamojiHenkan	処理13	checkString=argCheckData
zenKataKanamojiHenkan	処理14	int count=0
zenKataKanamojiHenkan	条件15	;count<Kananmoji[0].length;count++
zenKataKanamojiHenkan	処理16	int index=0
zenKataKanamojiHenkan	処理17	index=checkString.indexOf(Kananmoji[0][count])
zenKataKanamojiHenkan	条件18	index>-1
zenKataKanamojiHenkan	処理19	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenKataKanamojiHenkan	処理20	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])

イベント	参照番号	原本条件・処理
zenKataKanamojiHenkan	処理21	checkString=checkStringBuffer.toString()
zenKataKanamojiHenkan	処理22	s=s+" in="+index+" co="+count
zenkakumokiHenkan	条件1	;count<Kananmoji[0].length;count++
zenkakumokiHenkan	処理2	s=s+" in="+index+" co="+count
zenkakumokiHenkan	処理3	checkString=checkStringBuffer.toString()
zenkakumokiHenkan	処理4	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])
zenkakumokiHenkan	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenkakumokiHenkan	条件6	index>-1
zenkakumokiHenkan	処理7	index=checkString.indexOf(Kananmoji[0][count])
zenkakumokiHenkan	処理8	int index=0
zenkakumokiHenkan	処理9	int count=0
zenkakumokiHenkan	処理10	checkString=argCheckData
zenkakumokiHenkan	処理11	String checkString=""
zenkakumokiHenkan	処理12	String checkString=""
zenkakumokiHenkan	処理13	checkString=argCheckData
zenkakumokiHenkan	処理14	int count=0
zenkakumokiHenkan	条件15	;count<Kananmoji[0].length;count++
zenkakumokiHenkan	処理16	int index=0
zenkakumokiHenkan	処理17	index=checkString.indexOf(Kananmoji[0][count])

イベント	参照番号	原本条件・処理
zenkakumokiHenkan	条件18	index>-1
zenkakumokiHenkan	処理19	StringBuffer checkStringBuffer = new StringBuffer(checkString)
zenkakumokiHenkan	処理20	checkStringBuffer.replace(index,index+1,Kananmoji[1][count])
zenkakumokiHenkan	処理21	checkString=checkStringBuffer.toString()
zenkakumokiHenkan	処理22	s=s+" in="+index+" co="+count
decodeSmallzenKana	条件1	;count<smallzenKana[0].length;count++
decodeSmallzenKana	処理2	s=s+" in="+index+" co="+count
decodeSmallzenKana	処理3	checkString=checkStringBuffer.toString()
decodeSmallzenKana	処理4	checkStringBuffer.replace(index,index+1,smallzenKana[1][count])
decodeSmallzenKana	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallzenKana	条件6	index>-1
decodeSmallzenKana	処理7	index=checkString.indexOf(smallzenKana[0][count])
decodeSmallzenKana	処理8	int index=0
decodeSmallzenKana	処理9	int count=0
decodeSmallzenKana	処理10	checkString=argCheckData
decodeSmallzenKana	処理11	String checkString=""
decodeSmallzenKana	処理12	String checkString=""
decodeSmallzenKana	処理13	checkString=argCheckData
decodeSmallzenKana	処理14	int count=0

イベント	参照番号	原本条件・処理
decodeSmallzenKana	条件15	;count<smallzenKana[0].length;count++
decodeSmallzenKana	処理16	int index=0
decodeSmallzenKana	処理17	index=checkString.indexOf(smallzenKana[0][count])
decodeSmallzenKana	条件18	index>-1
decodeSmallzenKana	処理19	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallzenKana	処理20	checkStringBuffer.replace(index,index+1,smallzenKana[1][count])
decodeSmallzenKana	処理21	checkString=checkStringBuffer.toString()
decodeSmallzenKana	処理22	s=s+" in="+index+" co="+count
decodeSmallkatakana	条件1	;count<smallzenKana[0].length;count++
decodeSmallkatakana	処理2	s=s+" in="+index+" co="+count
decodeSmallkatakana	処理3	checkString=checkStringBuffer.toString()
decodeSmallkatakana	処理4	checkStringBuffer.replace(index,index+1,smallzenKana[1][count])
decodeSmallkatakana	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallkatakana	条件6	index>-1
decodeSmallkatakana	処理7	index=checkString.indexOf(smallzenKana[0][count])
decodeSmallkatakana	処理8	int index=0
decodeSmallkatakana	処理9	int count=0
decodeSmallkatakana	処理10	checkString=argCheckData
decodeSmallkatakana	処理11	String checkString=""

イベント	参照番号	原本条件・処理
decodeSmallkatakana	処理12	String checkString=""
decodeSmallkatakana	処理13	checkString=argCheckData
decodeSmallkatakana	処理14	int count=0
decodeSmallkatakana	条件15	;count<smallzenKana[0].length;count++
decodeSmallkatakana	処理16	int index=0
decodeSmallkatakana	処理17	index=checkString.indexOf(smallzenKana[0][count])
decodeSmallkatakana	条件18	index>-1
decodeSmallkatakana	処理19	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeSmallkatakana	処理20	checkStringBuffer.replace(index,index+1,smallzenKana[1][count])
decodeSmallkatakana	処理21	checkString=checkStringBuffer.toString()
decodeSmallkatakana	処理22	s=s+" in="+index+" co="+count
alphabetZenHenkan	条件1	;count<alphabet[0].length;count++
alphabetZenHenkan	処理2	s=s+" in="+index+" co="+count
alphabetZenHenkan	処理3	checkString=checkStringBuffer.toString()
alphabetZenHenkan	処理4	checkStringBuffer.replace(index,index+1,alphabet[1][count])
alphabetZenHenkan	処理5	StringBuffer checkStringBuffer = new StringBuffer(checkString)
alphabetZenHenkan	条件6	index>-1
alphabetZenHenkan	処理7	index=checkString.indexOf(alphabet[0][count])
alphabetZenHenkan	処理8	int index=0

イベント	参照番号	原本条件・処理
alphabetZenHenkan	処理9	int count=0
alphabetZenHenkan	処理10	checkString=argCheckData
alphabetZenHenkan	処理11	String checkString=""
alphabetZenHenkan	処理12	String[][] alphabet = {{" a "," b "," c "," d "," e "," f "," g "," h "," i "," j "," k "," l ","m"," n "," o ", " p "," q "," r "," s "," t "," u "," v ","w"," x "," y "," z "}, {" A "," B "," C "," D "," E "," F "," G "," H "," I "," J "," K "," L ","M"," N "," O "," P "," Q "," R "," S "," T "," U "," V ","W"," X "," Y "," Z "}, }
alphabetZenHenkan	処理13	String[][] alphabet = {{" a "," b "," c "," d "," e "," f "," g "," h "," i "," j "," k "," l ","m"," n "," o ", " p "," q "," r "," s "," t "," u "," v ","w"," x "," y "," z "}, {" A "," B "," C "," D "," E "," F "," G "," H "," I "," J "," K "," L ","M"," N "," O "," P "," Q "," R "," S "," T "," U "," V ","W"," X "," Y "," Z "}, }
alphabetZenHenkan	処理14	String checkString=""
alphabetZenHenkan	処理15	checkString=argCheckData
alphabetZenHenkan	処理16	int count=0
alphabetZenHenkan	条件17	;count<alphabet[0].length;count++
alphabetZenHenkan	処理18	int index=0
alphabetZenHenkan	処理19	index=checkString.indexOf(alphabet[0][count])
alphabetZenHenkan	条件20	index>-1
alphabetZenHenkan	処理21	StringBuffer checkStringBuffer = new StringBuffer(checkString)
alphabetZenHenkan	処理22	checkStringBuffer.replace(index,index+1,alphabet[1][count])
alphabetZenHenkan	処理23	checkString=checkStringBuffer.toString()

イベント	参照番号	原本条件・処理
alphabetZenHenkan	処理24	s=s+" in="+index+" co="+count
decodeLongMinus	処理1	checkString=checkStringBuffer.toString()
decodeLongMinus	処理2	giCnt=index
decodeLongMinus	処理3	checkStringBuffer.replace(index,index+1,q)
decodeLongMinus	処理4	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeLongMinus	処理5	int index=checkString.indexOf("?")
decodeLongMinus	処理6	String q="－"
decodeLongMinus	処理7	checkString=argCheckData
decodeLongMinus	処理8	String checkString
decodeLongMinus	処理9	String checkString
decodeLongMinus	処理10	checkString=argCheckData
decodeLongMinus	処理11	String q="－"
decodeLongMinus	処理12	int index=checkString.indexOf("?")
decodeLongMinus	処理13	StringBuffer checkStringBuffer = new StringBuffer(checkString)
decodeLongMinus	処理14	checkStringBuffer.replace(index,index+1,q)
decodeLongMinus	処理15	giCnt=index
decodeLongMinus	処理16	checkString=checkStringBuffer.toString()
editNewLine	処理1	s=sb.toString()
editNewLine	処理2	sb.replace(index,index+2,"　")

イベント	参照番号	原本条件・処理
editNewLine	処理3	StringBuffer sb=new StringBuffer(s)
editNewLine	処理4	int index=s.indexOf("\r\n")
editNewLine	処理5	String s=input
editNewLine	処理6	String s=input
editNewLine	処理7	int index=s.indexOf("\r\n")
editNewLine	処理8	StringBuffer sb=new StringBuffer(s)
editNewLine	処理9	sb.replace(index,index+2,"　")
editNewLine	処理10	s=sb.toString()
editKanma	処理1	editKingaku = fugou + editKingaku
editKanma	条件2	!fugou.equals("")
editKanma	条件3	; setCount > 0 ; setCount--
editKanma	処理4	leng++
editKanma	処理5	setPoint = setPoint + 4
editKanma	処理6	editKingaku = editKingaku.substring(0,setPoint) + "," + editKingaku.substring(setPoint,leng)
editKanma	処理7	setPoint = setPoint + 3
editKanma	処理8	int setPoint = leng % 3
editKanma	処理9	int setCount = (leng -1) / 3
editKanma	処理10	int leng = editKingaku.length()
editKanma	条件11	int cnt =editKingaku.length(); cnt > 1 ; cnt = cnt - 1

イベント	参照番号	原本条件・処理
editKanma	処理12	editKingaku = editKingaku.substring(1,cnt)
editKanma	処理13	cnt = 0
editKanma	条件14	int cnt =editKingaku.length(); cnt > 0 ; cnt = cnt - 9
editKanma	処理15	int top = Integer.parseInt(editKingaku.substring(startPoint,endPoint))
editKanma	処理16	startPoint = cnt - 9
editKanma	処理17	startPoint = 0
editKanma	条件18	cnt > 9
editKanma	処理19	endPoint = cnt
editKanma	処理20	int endPoint
editKanma	処理21	int startPoint
editKanma	処理22	editKingaku = editKingaku.substring(1,editKingaku.length())
editKanma	処理23	fugou = editKingaku.substring(0,1)
editKanma	処理24	int top = Integer.parseInt(editKingaku.substring(0,1))
editKanma	処理25	String fugou =""
editKanma	処理26	String editKingaku = kingaku
editKanma	処理27	String editKingaku = kingaku
editKanma	処理28	String fugou =""
editKanma	処理29	int top = Integer.parseInt(editKingaku.substring(0,1))
editKanma	処理30	fugou = editKingaku.substring(0,1)

イベント	参照番号	原本条件・処理
editKanma	処理31	editKingaku = editKingaku.substring(1,editKingaku.length())
editKanma	処理32	int startPoint
editKanma	処理33	int endPoint
editKanma	条件34	int cnt =editKingaku.length(); cnt > 0 ; cnt = cnt - 9
editKanma	処理35	endPoint = cnt
editKanma	条件36	cnt > 9
editKanma	処理37	startPoint = cnt - 9
editKanma	処理38	startPoint = 0
editKanma	処理39	int top = Integer.parseInt(editKingaku.substring(startPoint,endPoint))
editKanma	条件40	int cnt =editKingaku.length(); cnt > 1 ; cnt = cnt - 1
editKanma	処理41	editKingaku = editKingaku.substring(1,cnt)
editKanma	処理42	cnt = 0
editKanma	処理43	int leng = editKingaku.length()
editKanma	処理44	int setCount = (leng -1) / 3
editKanma	処理45	int setPoint = leng % 3
editKanma	処理46	setPoint = setPoint + 3
editKanma	条件47	; setCount > 0 ; setCount--
editKanma	処理48	editKingaku = editKingaku.substring(0,setPoint) + "," + editKingaku.substring(setPoint,leng)

イベント	参照番号	原本条件・処理
editKanma	処理49	setPoint = setPoint + 4
editKanma	処理50	leng++
editKanma	条件51	!fugou.equals("")
editKanma	処理52	editKingaku = fugou + editKingaku
editKanma	条件53	int cnt =editKingaku.length(); cnt > 0 ; cnt = cnt - 9
editKanma	条件54	NumberFormatException nfe
editTelphone	例外	telphone.length()
editTelphone	処理2	String editTel = ""
editTelphone	処理3	String editTel = ""
editDate	処理1	dateString = dateStringBuffer.toString()
editDate	処理2	StringBuffer dateStringBuffer = new StringBuffer(eDate)
editDate	処理3	String dateString=""
editDate	処理4	dateString = dateStringBuffer.toString()
editDate	処理5	StringBuffer dateStringBuffer = new StringBuffer(eDate)
editDate	処理6	String dateString=""
editDate	処理7	String dateString=""
editDate	処理8	StringBuffer dateStringBuffer = new StringBuffer(eDate)
editDate	処理9	dateString = dateStringBuffer.toString()
editDate	処理10	String dateString=""

イベント	参照番号	原本条件・処理
editDate	処理11	StringBuffer dateStringBuffer = new StringBuffer(eDate)
editDate	処理12	dateString = dateStringBuffer.toString()
edit49Date	処理1	StringBuffer sbDate=new StringBuffer(eDate)
edit49Date	処理2	StringBuffer sbDate=new StringBuffer(eDate)
editTime	処理1	timeString = timeStringBuffer.toString()
editTime	処理2	StringBuffer timeStringBuffer = new StringBuffer(eTime)
editTime	処理3	String timeString =""
editTime	処理4	String timeString =""
editTime	処理5	StringBuffer timeStringBuffer = new StringBuffer(eTime)
editTime	処理6	timeString = timeStringBuffer.toString()
display	処理1	out.println("</tr></table>")
display	処理2	out.println(ex.getMessage())
display	処理3	out.println("</table>")
display	条件4	rs.next()
display	処理5	int row = 0
display	処理6	out.println("</table>")
display	条件7	rs.next()
display	処理8	out.println("</tr>")
display	条件9	int i = 1; i <= columns; i++

イベント	参照番号	原本条件・処理
display	処理10	out.print("<th>" + rsmd.getColumnTypeName(i) + "</th>")
display	処理11	out.println("</tr>")
display	条件12	int i = 1; i <= columns; i++
display	処理13	out.print("<th>" + rsmd.getColumnName(i) + "</th>")
display	処理14	int columns = rsmd.getColumnCount()
display	処理15	ResultSetMetaData rsmd = rs.getMetaData()
display	処理16	out.print("<tr>")
display	処理17	out.println("<table border>")
display	処理18	out.println("<table border>")
display	処理19	out.print("<tr>")
display	処理20	ResultSetMetaData rsmd = rs.getMetaData()
display	処理21	int columns = rsmd.getColumnCount()
display	条件22	int i = 1; i <= columns; i++
display	処理23	out.print("<th>" + rsmd.getColumnName(i) + "</th>")
display	処理24	out.println("</tr>")
display	条件25	int i = 1; i <= columns; i++
display	処理26	out.print("<th>" + rsmd.getColumnTypeName(i) + "</th>")
display	処理27	out.println("</tr>")
display	条件28	rs.next()

イベント	参照番号	原本条件・処理
display	処理29	int row = 0
display	条件30	rs.next()
display	処理31	out.println("</table>")
display	処理32	out.println("</table>")
display	処理33	out.println(ex.getMessage())
display	処理34	out.println("</tr></table>")
getSeireki	処理1	boolean check = checkDate(seirekiYear,seirekiMonth,seirekiDay)
getSeireki	処理2	seirekiDay = Integer.parseInt(warekiDay)
getSeireki	処理3	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理4	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki	処理5	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理6	seirekiYear = 1925 + Integer.parseInt(warekiYear)
getSeireki	処理7	seirekiYear = 1911 + Integer.parseInt(warekiYear)
getSeireki	処理8	seirekiYear = 1867 + Integer.parseInt(warekiYear)
getSeireki	処理9	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	条件10	gengo.equals("M")
getSeireki	条件11	gengo.equals("T")
getSeireki	条件12	gengo.equals("S")
getSeireki	条件13	gengo.equals("H")

イベント	参照番号	原本条件・処理
getSeireki	処理14	warekiDay =wareki.substring(5,7)
getSeireki	処理15	warekiMonth =wareki.substring(3,5)
getSeireki	処理16	warekiYear =wareki.substring(1,3)
getSeireki	処理17	gengo =wareki.substring(0,1)
getSeireki	処理18	seirekiDay = Integer.parseInt(warekiDay)
getSeireki	処理19	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理20	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki	処理21	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理22	warekiDay = wareki.substring(4,6)
getSeireki	処理23	warekiMonth = wareki.substring(2,4)
getSeireki	処理24	warekiYear = wareki.substring(0,2)
getSeireki	処理25	seirekiDay = 10
getSeireki	処理26	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理27	seirekiDate = Integer.toString(seirekiYear) + warekiMonth
getSeireki	処理28	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理29	warekiMonth = wareki.substring(2,4)
getSeireki	処理30	warekiYear = wareki.substring(0,2)
getSeireki	処理31	seirekiDay = 10
getSeireki	処理32	seirekiMonth = 1

イベント	参照番号	原本条件・処理
getSeireki	処理33	seirekiDate = Integer.toString(seirekiYear)
getSeireki	処理34	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理35	warekiYear = wareki.substring(0,2)
getSeireki	処理36	int wk = Integer.parseInt(wareki)
getSeireki	処理37	int seirekiDay
getSeireki	処理38	int seirekiMonth
getSeireki	処理39	int seirekiYear
getSeireki	処理40	String warekiDay = ""
getSeireki	処理41	String warekiMonth = ""
getSeireki	処理42	String warekiYear = ""
getSeireki	処理43	String gengo=""
getSeireki	処理44	String seirekiDate = ""
getSeireki	処理45	String seirekiDate = ""
getSeireki	処理46	String gengo=""
getSeireki	処理47	String warekiYear = ""
getSeireki	処理48	String warekiMonth = ""
getSeireki	処理49	String warekiDay = ""
getSeireki	処理50	int seirekiYear
getSeireki	処理51	int seirekiMonth

イベント	参照番号	原本条件・処理
getSeireki	処理52	int seirekiDay
getSeireki	処理53	gengo =wareki.substring(0,1)
getSeireki	処理54	warekiYear =wareki.substring(1,3)
getSeireki	処理55	warekiMonth =wareki.substring(3,5)
getSeireki	処理56	warekiDay =wareki.substring(5,7)
getSeireki	条件57	gengo.equals("H")
getSeireki	処理58	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	条件59	gengo.equals("S")
getSeireki	処理60	seirekiYear = 1925 + Integer.parseInt(warekiYear)
getSeireki	条件61	gengo.equals("T")
getSeireki	処理62	seirekiYear = 1911 + Integer.parseInt(warekiYear)
getSeireki	条件63	gengo.equals("M")
getSeireki	処理64	seirekiYear = 1867 + Integer.parseInt(warekiYear)
getSeireki	処理65	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理66	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki	処理67	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理68	seirekiDay = Integer.parseInt(warekiDay)
getSeireki	処理69	int wk = Integer.parseInt(wareki)
getSeireki	処理70	warekiYear = wareki.substring(0,2)

イベント	参照番号	原本条件・処理
getSeireki	処理71	warekiMonth = wareki.substring(2,4)
getSeireki	処理72	warekiDay = wareki.substring(4,6)
getSeireki	処理73	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理74	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki	処理75	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理76	seirekiDay = Integer.parseInt(warekiDay)
getSeireki	処理77	warekiYear = wareki.substring(0,2)
getSeireki	処理78	warekiMonth = wareki.substring(2,4)
getSeireki	処理79	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理80	seirekiDate = Integer.toString(seirekiYear) + warekiMonth
getSeireki	処理81	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki	処理82	seirekiDay = 10
getSeireki	処理83	warekiYear = wareki.substring(0,2)
getSeireki	処理84	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki	処理85	seirekiDate = Integer.toString(seirekiYear)
getSeireki	処理86	seirekiMonth = 1
getSeireki	処理87	seirekiDay = 10
getSeireki	処理88	boolean check = checkDate(seirekiYear,seirekiMonth,seirekiDay)
getSeireki	条件89	NumberFormatException nfe

イベント	参照番号	原本条件・処理
checkDate	処理1	String day = seireki.substring(seireki.length() - 2,seireki.length())
checkDate	処理2	String month = seireki.substring(seireki.length() - 4,seireki.length() - 2)
checkDate	処理3	String year = seireki.substring(0,seireki.length() - 4)
checkDate	条件4	seireki.length() < 5
checkDate	条件5	seireki.length() < 5
checkDate	処理6	String year = seireki.substring(0,seireki.length() - 4)
checkDate	処理7	String month = seireki.substring(seireki.length() - 4,seireki.length() - 2)
checkDate	処理8	String day = seireki.substring(seireki.length() - 2,seireki.length())
checkDate	条件9	seireki.length() < 5
checkDate	条件10	NOT (seireki.length() < 5)
checkDate	処理1	check = false
checkDate	処理2	java.util.Date ud = cal.getTime()
checkDate	処理3	cal.set(year, month -1, day)
checkDate	処理4	boolean check = true
checkDate	処理5	boolean check = true
checkDate	処理6	cal.set(year, month -1, day)
checkDate	処理7	java.util.Date ud = cal.getTime()
checkDate	処理8	check = false

イベント	参照番号	原本条件・処理
checkWarekiDate	処理1	status=true
checkWarekiDate	条件2	!getSeireki(wareki).equals("PARAMERR")
checkWarekiDate	処理3	int wk=Integer.parseInt(date)
checkWarekiDate	処理4	date=new String(wareki)
checkWarekiDate	処理5	date=new String(wareki.substring(1,7))
checkWarekiDate	条件6	gengo.equals("H")|gengo.equals("S")|gengo.equals("T")| gengo.equals("M")
checkWarekiDate	処理7	String gengo=new String(wareki.substring(0,1))
checkWarekiDate	処理8	String date
checkWarekiDate	処理9	boolean status=false
checkWarekiDate	処理10	boolean status=false
checkWarekiDate	処理11	String date
checkWarekiDate	処理12	date=new String(wareki)
checkWarekiDate	処理13	String gengo=new String(wareki.substring(0,1))
checkWarekiDate	条件14	gengo.equals("H")|gengo.equals("S")|gengo.equals("T")| gengo.equals("M")
checkWarekiDate	処理15	date=new String(wareki.substring(1,7))
checkWarekiDate	処理16	int wk=Integer.parseInt(date)
checkWarekiDate	条件17	!getSeireki(wareki).equals("PARAMERR")
checkWarekiDate	処理18	status=true

イベント	参照番号	原本条件・処理
checkWarekiDate	条件19	NOT (gengo.equals("H")|gengo.equals("S")|gengo.equals("T")|gengo.equals("M"))
checkWarekiDate	条件20	NumberFormatException nfe
getSanissu	処理1	long nissu = (endTime - startTime)/(10006060*24)
getSanissu	処理2	long endTime = (cal.getTime()).getTime()
getSanissu	処理3	cal.set(endYear, endMonth - 1, endDay)
getSanissu	処理4	cal.clear()
getSanissu	処理5	long startTime= (cal.getTime()).getTime()
getSanissu	処理6	cal.set(startYear, startMonth - 1, startDay)
getSanissu	処理7	cal.clear()
getSanissu	処理8	check = checkDate(endYear, endMonth, endDay)
getSanissu	処理9	check = checkDate(startYear, startMonth, startDay)
getSanissu	処理10	boolean check = true
getSanissu	処理11	boolean check = true
getSanissu	処理12	check = checkDate(startYear, startMonth, startDay)
getSanissu	処理13	check = checkDate(endYear, endMonth, endDay)
getSanissu	処理14	cal.clear()
getSanissu	処理15	cal.set(startYear, startMonth - 1, startDay)
getSanissu	処理16	long startTime= (cal.getTime()).getTime()
getSanissu	処理17	cal.clear()

イベント	参照番号	原本条件・処理
getSanissu	処理18	cal.set(endYear, endMonth - 1, endDay)
getSanissu	処理19	long endTime = (cal.getTime()).getTime()
getSanissu	処理20	long nissu = (endTime - startTime)/(10006060*24)
getSanissu	処理1	wk = Integer.parseInt(endDay)
getSanissu	処理2	wk = Integer.parseInt(endMonth)
getSanissu	処理3	int wk = Integer.parseInt(endYear)
getSanissu	処理4	wk = Integer.parseInt(startDay)
getSanissu	処理5	wk = Integer.parseInt(startMonth)
getSanissu	処理6	int wk = Integer.parseInt(startYear)
getSanissu	処理7	int wk = Integer.parseInt(startYear)
getSanissu	処理8	wk = Integer.parseInt(startMonth)
getSanissu	処理9	wk = Integer.parseInt(startDay)
getSanissu	処理10	int wk = Integer.parseInt(endYear)
getSanissu	処理11	wk = Integer.parseInt(endMonth)
getSanissu	処理12	wk = Integer.parseInt(endDay)
getSanissu	条件13	NumberFormatException nfe
getSanissu	条件14	NumberFormatException nfe
getSanissu	処理1	String endDay = endDate.substring(endDate.length() - 2,endDate.length())

イベント	参照番号	原本条件・処理
getSanissu	処理2	String endMonth = endDate.substring(endDate.length() - 4,endDate.length() - 2)
getSanissu	処理3	String endYear = endDate.substring(0,endDate.length() - 4)
getSanissu	処理4	String startDay = startDate.substring(startDate.length() - 2,startDate.length())
getSanissu	処理5	String startMonth = startDate.substring(startDate.length() - 4,startDate.length() - 2)
getSanissu	処理6	String startYear = startDate.substring(0,startDate.length() - 4)
getSanissu	処理7	String startYear = startDate.substring(0,startDate.length() - 4)
getSanissu	処理8	String startMonth = startDate.substring(startDate.length() - 4,startDate.length() - 2)
getSanissu	処理9	String startDay = startDate.substring(startDate.length() - 2,startDate.length())
getSanissu	処理10	String endYear = endDate.substring(0,endDate.length() - 4)
getSanissu	処理11	String endMonth = endDate.substring(endDate.length() - 4,endDate.length() - 2)
getSanissu	処理12	String endDay = endDate.substring(endDate.length() - 2,endDate.length())
getSanissu	条件1	endDate8keta.length() < 8
getSanissu	処理2	endDate8keta = "0"+ endDate8keta
getSanissu	処理3	String endDate8keta = Integer.toString(endDate)
getSanissu	条件4	startDate8keta.length() < 8

イベント	参照番号	原本条件・処理
getSanissu	処理5	startDate8keta = "0"+ startDate8keta
getSanissu	処理6	String startDate8keta = Integer.toString(startDate)
getSanissu	処理7	String startDate8keta = Integer.toString(startDate)
getSanissu	条件8	startDate8keta.length() < 8
getSanissu	処理9	startDate8keta = "0"+ startDate8keta
getSanissu	処理10	String endDate8keta = Integer.toString(endDate)
getSanissu	条件11	endDate8keta.length() < 8
getSanissu	処理12	endDate8keta = "0"+ endDate8keta
getAge	条件1	endDate8keta.length() < 8
getAge	処理2	endDate8keta = "0"+ endDate8keta
getAge	処理3	String endDate8keta = Integer.toString(endDate)
getAge	条件4	startDate8keta.length() < 8
getAge	処理5	startDate8keta = "0"+ startDate8keta
getAge	処理6	String startDate8keta = Integer.toString(startDate)
getAge	処理7	String startDate8keta = Integer.toString(startDate)
getAge	条件8	startDate8keta.length() < 8
getAge	処理9	startDate8keta = "0"+ startDate8keta
getAge	処理10	String endDate8keta = Integer.toString(endDate)
getAge	条件11	endDate8keta.length() < 8

イベント	参照番号	原本条件・処理
getAge	処理12	endDate8keta = "0"+ endDate8keta
getAge	処理1	String endDay = endDate.substring(endDate.length() - 2,endDate.length())
getAge	処理2	String endMonth = endDate.substring(endDate.length() - 4,endDate.length() - 2)
getAge	処理3	String endYear = endDate.substring(0,endDate.length() - 4)
getAge	処理4	String startDay = startDate.substring(startDate.length() - 2,startDate.length())
getAge	処理5	String startMonth = startDate.substring(startDate.length() - 4,startDate.length() - 2)
getAge	処理6	String startYear = startDate.substring(0,startDate.length() - 4)
getAge	処理7	String startYear = startDate.substring(0,startDate.length() - 4)
getAge	処理8	String startMonth = startDate.substring(startDate.length() - 4,startDate.length() - 2)
getAge	処理9	String startDay = startDate.substring(startDate.length() - 2,startDate.length())
getAge	処理10	String endYear = endDate.substring(0,endDate.length() - 4)
getAge	処理11	String endMonth = endDate.substring(endDate.length() - 4,endDate.length() - 2)
getAge	処理12	String endDay = endDate.substring(endDate.length() - 2,endDate.length())
getAge	処理1	wk = Integer.parseInt(endDay)
getAge	処理2	wk = Integer.parseInt(endMonth)

イベント	参照番号	原本条件・処理
getAge	処理3	int wk = Integer.parseInt(endYear)
getAge	処理4	wk = Integer.parseInt(startDay)
getAge	処理5	wk = Integer.parseInt(startMonth)
getAge	処理6	int wk = Integer.parseInt(startYear)
getAge	処理7	int wk = Integer.parseInt(startYear)
getAge	処理8	wk = Integer.parseInt(startMonth)
getAge	処理9	wk = Integer.parseInt(startDay)
getAge	処理10	int wk = Integer.parseInt(endYear)
getAge	処理11	wk = Integer.parseInt(endMonth)
getAge	処理12	wk = Integer.parseInt(endDay)
getAge	条件13	NumberFormatException nfe
getAge	条件14	NumberFormatException nfe
getAge	処理1	age--
getAge	処理2	age--
getAge	条件3	endDay < startDay
getAge	条件4	endMonth < startMonth
getAge	処理5	int age = endYear - startYear
getAge	条件6	endDay < startDay
getAge	条件7	endMonth < startMonth

イベント	参照番号	原本条件・処理
getAge	条件8	endYear < startYear
getAge	処理9	check = checkDate(endYear, endMonth, endDay)
getAge	処理10	check = checkDate(startYear, startMonth, startDay)
getAge	処理11	boolean check = true
getAge	条件12	endDay8keta.length() < 2
getAge	処理13	endDay8keta = "0"+ endDay8keta
getAge	処理14	String endDay8keta = Integer.toString(endDay)
getAge	条件15	endMonth8keta.length() < 2
getAge	処理16	endMonth8keta = "0"+ endMonth8keta
getAge	処理17	String endMonth8keta = Integer.toString(endMonth)
getAge	条件18	endYear8keta.length() < 4
getAge	処理19	endYear8keta = "0"+ endYear8keta
getAge	処理20	String endYear8keta = Integer.toString(endYear)
getAge	条件21	startDay8keta.length() < 2
getAge	処理22	startDay8keta = "0"+ startDay8keta
getAge	処理23	String startDay8keta = Integer.toString(startDay)
getAge	条件24	startMonth8keta.length() < 2
getAge	処理25	startMonth8keta = "0"+ startMonth8keta
getAge	処理26	String startMonth8keta = Integer.toString(startMonth)

イベント	参照番号	原本条件・処理
getAge	条件27	startYear8keta.length() < 4
getAge	処理28	startYear8keta = "0"+ startYear8keta
getAge	処理29	String startYear8keta = Integer.toString(startYear)
getAge	処理30	String startYear8keta = Integer.toString(startYear)
getAge	条件31	startYear8keta.length() < 4
getAge	処理32	startYear8keta = "0"+ startYear8keta
getAge	処理33	String startMonth8keta = Integer.toString(startMonth)
getAge	条件34	startMonth8keta.length() < 2
getAge	処理35	startMonth8keta = "0"+ startMonth8keta
getAge	処理36	String startDay8keta = Integer.toString(startDay)
getAge	条件37	startDay8keta.length() < 2
getAge	処理38	startDay8keta = "0"+ startDay8keta
getAge	処理39	String endYear8keta = Integer.toString(endYear)
getAge	条件40	endYear8keta.length() < 4
getAge	処理41	endYear8keta = "0"+ endYear8keta
getAge	処理42	String endMonth8keta = Integer.toString(endMonth)
getAge	条件43	endMonth8keta.length() < 2
getAge	処理44	endMonth8keta = "0"+ endMonth8keta
getAge	処理45	String endDay8keta = Integer.toString(endDay)

イベント	参照番号	原本条件・処理
getAge	条件46	endDay8keta.length() < 2
getAge	処理47	endDay8keta = "0"+ endDay8keta
getAge	処理48	boolean check = true
getAge	処理49	check = checkDate(startYear, startMonth, startDay)
getAge	処理50	check = checkDate(endYear, endMonth, endDay)
getAge	条件51	endYear < startYear
getAge	条件52	endMonth < startMonth
getAge	条件53	endDay < startDay
getAge	処理54	int age = endYear - startYear
getAge	条件55	endMonth < startMonth
getAge	処理56	age--
getAge	条件57	endDay < startDay
getAge	処理58	age--
getAge	条件59	endYear < startYear
getAge	条件60	NOT (endYear < startYear)
getAge	条件61	endMonth < startMonth
getAge	条件62	NOT (endYear < startYear)
getAge	条件63	NOT (endMonth < startMonth)
getAge	条件64	endDay < startDay

イベント	参照番号	原本条件・処理
getAge	条件65	NOT (endYear < startYear)
checkEigyobi	処理1	int youbi= cal.get(Calendar.DAY_OF_WEEK)
checkEigyobi	処理2	cal.set(year, month -1, day)
checkEigyobi	処理3	int day = Integer.parseInt(seireki.substring(seireki.length() -2,seireki.length()))
checkEigyobi	処理4	int month = Integer.parseInt(seireki.substring(seireki.length() -4,seireki.length() - 2))
checkEigyobi	処理5	int year = Integer.parseInt(seireki.substring(0,seireki.length() - 4))
checkEigyobi	条件6	!checkDate(seireki)
checkEigyobi	処理7	int wk = Integer.parseInt(seireki)
checkEigyobi	処理8	int wk = Integer.parseInt(seireki)
checkEigyobi	条件9	!checkDate(seireki)
checkEigyobi	処理10	int year = Integer.parseInt(seireki.substring(0,seireki.length() - 4))
checkEigyobi	処理11	int month = Integer.parseInt(seireki.substring(seireki.length() -4,seireki.length() - 2))
checkEigyobi	処理12	int day = Integer.parseInt(seireki.substring(seireki.length() -2,seireki.length()))
checkEigyobi	処理13	cal.set(year, month -1, day)
checkEigyobi	処理14	int youbi= cal.get(Calendar.DAY_OF_WEEK)
checkEigyobi	条件15	NumberFormatException nfe
checkEigyobi	条件16	!checkDate(seireki)

イベント	参照番号	原本条件・処理
checkEigyobi	条件17	NOT (!checkDate(seireki))
checkEigyobi	条件18	NOT (!checkDate(seireki))
checkEigyobi	条件19	NOT (RqsDbSearch.checkHoliday(seireki))
checkEigyobi	条件20	NOT (!checkDate(seireki))
checkEigyobi	条件21	NOT (RqsDbSearch.checkHoliday(seireki))
getSanissuDate	条件1	SanissuDateDay.length() < 2
getSanissuDate	処理2	SanissuDateDay = "0" + SanissuDateDay
getSanissuDate	条件3	SanissuDateMonth.length() < 2
getSanissuDate	処理4	SanissuDateMonth = "0" + SanissuDateMonth
getSanissuDate	処理5	String SanissuDateDay = Integer.toString(day)
getSanissuDate	処理6	String SanissuDateMonth = Integer.toString(month)
getSanissuDate	処理7	String SanissuDateYear = Integer.toString(year)
getSanissuDate	処理8	month++
getSanissuDate	処理9	day = cal.get(Calendar.DATE)
getSanissuDate	処理10	month = cal.get(Calendar.MONTH)
getSanissuDate	処理11	year = cal.get(Calendar.YEAR)
getSanissuDate	処理12	cal.add(Calendar.DATE,sanissu)
getSanissuDate	処理13	cal.set(year, month -1, day)
getSanissuDate	処理14	int day = Integer.parseInt(seireki.substring(seireki.length() -2,seireki.length()))

イベント	参照番号	原本条件・処理
getSanissuDate	処理15	int month = Integer.parseInt(seireki.substring(seireki.length() - 4,seireki.length() - 2))
getSanissuDate	処理16	int year = Integer.parseInt(seireki.substring(0,seireki.length() - 4))
getSanissuDate	条件17	!checkDate(seireki)
getSanissuDate	処理18	int wk = Integer.parseInt(seireki)
getSanissuDate	処理19	int wk = Integer.parseInt(seireki)
getSanissuDate	条件20	!checkDate(seireki)
getSanissuDate	処理21	int year = Integer.parseInt(seireki.substring(0,seireki.length() - 4))
getSanissuDate	処理22	int month = Integer.parseInt(seireki.substring(seireki.length() - 4,seireki.length() - 2))
getSanissuDate	処理23	int day = Integer.parseInt(seireki.substring(seireki.length() - 2,seireki.length()))
getSanissuDate	処理24	cal.set(year, month -1, day)
getSanissuDate	処理25	cal.add(Calendar.DATE,sanissu)
getSanissuDate	処理26	year = cal.get(Calendar.YEAR)
getSanissuDate	処理27	month = cal.get(Calendar.MONTH)
getSanissuDate	処理28	day = cal.get(Calendar.DATE)
getSanissuDate	処理29	month++
getSanissuDate	処理30	String SanissuDateYear = Integer.toString(year)
getSanissuDate	処理31	String SanissuDateMonth = Integer.toString(month)
getSanissuDate	処理32	String SanissuDateDay = Integer.toString(day)

イベント	参照番号	原本条件・処理
getSanissuDate	条件33	SanissuDateMonth.length() < 2
getSanissuDate	処理34	SanissuDateMonth = "0" + SanissuDateMonth
getSanissuDate	条件35	SanissuDateDay.length() < 2
getSanissuDate	処理36	SanissuDateDay = "0" + SanissuDateDay
getSanissuDate	条件37	NumberFormatException nfe
getSanissuDate	条件38	!checkDate(seireki)
getSanissuDate	条件39	NOT (!checkDate(seireki))
getSanissuDate	条件40	NOT (!checkDate(seireki))
getSaeigyoubi	条件1	int i = 0; i < eigyoubiCheckCounter ; i++
getSaeigyoubi	条件2	!checkEigyobi(saeigyoubi)
getSaeigyoubi	処理3	sanissuParameter = -1
getSaeigyoubi	処理4	eigyoubiCheckCounter = eigyoubiCheckCounter * -1
getSaeigyoubi	条件5	saeigyoubisu < 0
getSaeigyoubi	処理6	String saeigyoubi = seireki
getSaeigyoubi	処理7	int sanissuParameter = 1
getSaeigyoubi	処理8	int eigyoubiCheckCounter = saeigyoubisu
getSaeigyoubi	条件9	!checkDate(seireki)
getSaeigyoubi	処理10	int wk = Integer.parseInt(seireki)
getSaeigyoubi	処理11	int wk = Integer.parseInt(seireki)

イベント	参照番号	原本条件・処理
getSaeigyoubi	条件12	!checkDate(seireki)
getSaeigyoubi	処理13	int eigyoubiCheckCounter = saeigyoubisu
getSaeigyoubi	処理14	int sanissuParameter = 1
getSaeigyoubi	処理15	String saeigyoubi = seireki
getSaeigyoubi	条件16	saeigyoubisu < 0
getSaeigyoubi	処理17	eigyoubiCheckCounter = eigyoubiCheckCounter * -1
getSaeigyoubi	処理18	sanissuParameter = -1
getSaeigyoubi	条件19	int i = 0; i < eigyoubiCheckCounter ; i++
getSaeigyoubi	条件20	!checkEigyobi(saeigyoubi)
getSaeigyoubi	条件21	NumberFormatException nfe
getSaeigyoubi	条件22	!checkDate(seireki)
getSaeigyoubi	条件23	NOT (!checkDate(seireki))
getSaeigyoubi	条件24	NOT (!checkDate(seireki))
getSeirekiForWareki	処理1	seireki = String.valueOf(18670000 + date)
getSeirekiForWareki	処理2	seireki = String.valueOf(19110000 + date)
getSeirekiForWareki	処理3	seireki = String.valueOf(19250000 + date)
getSeirekiForWareki	処理4	seireki = String.valueOf(19880000 + date)
getSeirekiForWareki	処理5	seireki = ""
getSeirekiForWareki	処理6	String seireki

イベント	参照番号	原本条件・処理
getSeirekiForWareki	条件7	date < 10000
getSeirekiForWareki	処理8	int date = Integer.parseInt(nen + tuki + niti)
getSeirekiForWareki	処理9	String niti = wareki.substring(8,10)
getSeirekiForWareki	処理10	String tuki = wareki.substring(5,7)
getSeirekiForWareki	処理11	String nen = wareki.substring(2,4)
getSeirekiForWareki	処理12	String gengo = wareki.substring(0,2)
getSeirekiForWareki	処理13	String gengo = wareki.substring(0,2)
getSeirekiForWareki	処理14	String nen = wareki.substring(2,4)
getSeirekiForWareki	処理15	String tuki = wareki.substring(5,7)
getSeirekiForWareki	処理16	String niti = wareki.substring(8,10)
getSeirekiForWareki	処理17	int date = Integer.parseInt(nen + tuki + niti)
getSeirekiForWareki	条件18	date < 10000
getSeirekiForWareki	処理19	String seireki
getSeirekiForWareki	処理20	seireki = String.valueOf(18670000 + date)
getSeirekiForWareki	処理21	seireki = String.valueOf(19110000 + date)
getSeirekiForWareki	処理22	seireki = String.valueOf(19250000 + date)
getSeirekiForWareki	処理23	seireki = String.valueOf(19880000 + date)
getSeirekiForWareki	処理24	seireki = ""
getSeirekiForWareki	条件25	date < 10000

イベント	参照番号	原本条件・処理
getSeirekiForWareki	条件26	NOT (date < 10000)
getSeirekiForWareki	条件27	NumberFormatException e
convertWareki2	処理1	wareki = wareki + "0"
convertWareki2	条件2	year < 10
convertWareki2	処理3	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable2[i][1].substring(0,4)) + 1
convertWareki2	処理4	wareki = gengoTable2[i][0]
convertWareki2	処理5	String wareki = ""
convertWareki2	条件6	; i < gengoTable2.length;i++
convertWareki2	処理7	break
convertWareki2	条件8	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable2[i][1])
convertWareki2	条件9	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable2[i][2])
convertWareki2	処理10	int i = 0
convertWareki2	処理11	Integer.parseInt(seireki)
convertWareki2	処理12	Integer.parseInt(seireki)
convertWareki2	処理13	int i = 0
convertWareki2	条件14	; i < gengoTable2.length;i++
convertWareki2	条件15	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable2[i][1])
convertWareki2	条件16	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable2[i][2])
convertWareki2	処理17	break

イベント	参照番号	原本条件・処理
convertWareki2	処理18	String wareki = ""
convertWareki2	処理19	wareki = gengoTable2[i][0]
convertWareki2	処理20	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable2[i][1].substring(0,4)) + 1
convertWareki2	条件21	year < 10
convertWareki2	処理22	wareki = wareki + "0"
convertWareki2	条件23	NumberFormatException nfe
convertSimpleWareki2	処理1	sbDate=new StringBuffer("")
convertSimpleWareki2	処理2	sbDate.replace(0,2,"R")
convertSimpleWareki2	処理3	sbDate.replace(0,2,"H")
convertSimpleWareki2	処理4	sbDate.replace(0,2,"S")
convertSimpleWareki2	処理5	sbDate.replace(0,2,"T")
convertSimpleWareki2	処理6	sbDate.replace(0,2,"M")
convertSimpleWareki2	処理7	sbDate=new StringBuffer(convertWareki2(seireki))
convertSimpleWareki2	処理8	StringBuffer sbDate
convertSimpleWareki2	処理9	StringBuffer sbDate
convertSimpleWareki2	処理10	sbDate=new StringBuffer("")
convertSimpleWareki2	処理11	sbDate=new StringBuffer(convertWareki2(seireki))
convertSimpleWareki2	処理12	sbDate.replace(0,2,"R")
convertSimpleWareki2	処理13	sbDate.replace(0,2,"H")

イベント	参照番号	原本条件・処理
convertSimpleWareki2	処理14	sbDate.replace(0,2,"S")
convertSimpleWareki2	処理15	sbDate.replace(0,2,"T")
convertSimpleWareki2	処理16	sbDate.replace(0,2,"M")
convertSimpleWareki2	条件17	Exception e
convertSeireki2	条件1	wareki.equals(convertSimpleWareki2(seireki))
convertSeireki2	処理2	seireki = String.valueOf(18670000 + date)
convertSeireki2	処理3	seireki = String.valueOf(19110000 + date)
convertSeireki2	処理4	seireki = String.valueOf(19250000 + date)
convertSeireki2	処理5	seireki = String.valueOf(19880000 + date)
convertSeireki2	処理6	seireki = String.valueOf(20180000 + date)
convertSeireki2	処理7	seireki = ""
convertSeireki2	条件8	gengo.equals("R")
convertSeireki2	条件9	gengo.equals("H")
convertSeireki2	条件10	gengo.equals("S")
convertSeireki2	条件11	gengo.equals("T")
convertSeireki2	条件12	gengo.equals("M")
convertSeireki2	処理13	String seireki
convertSeireki2	条件14	date < 10000
convertSeireki2	処理15	int date = Integer.parseInt(wareki.substring(1))

イベント	参照番号	原本条件・処理
convertSeireki2	処理16	String gengo = wareki.substring(0,1)
convertSeireki2	処理17	String gengo = wareki.substring(0,1)
convertSeireki2	処理18	int date = Integer.parseInt(wareki.substring(1))
convertSeireki2	条件19	date < 10000
convertSeireki2	処理20	String seireki
convertSeireki2	条件21	gengo.equals("M")
convertSeireki2	処理22	seireki = String.valueOf(18670000 + date)
convertSeireki2	条件23	gengo.equals("T")
convertSeireki2	処理24	seireki = String.valueOf(19110000 + date)
convertSeireki2	条件25	gengo.equals("S")
convertSeireki2	処理26	seireki = String.valueOf(19250000 + date)
convertSeireki2	条件27	gengo.equals("H")
convertSeireki2	処理28	seireki = String.valueOf(19880000 + date)
convertSeireki2	条件29	gengo.equals("R")
convertSeireki2	処理30	seireki = String.valueOf(20180000 + date)
convertSeireki2	処理31	seireki = ""
convertSeireki2	条件32	wareki.equals(convertSimpleWareki2(seireki))
convertSeireki2	条件33	date < 10000
convertSeireki2	条件34	NOT (date < 10000)

イベント	参照番号	原本条件・処理
convertSeireki2	条件35	wareki.equals(convertSimpleWareki2(seireki))
convertSeireki2	条件36	NOT (date < 10000)
convertSeireki2	条件37	NOT (wareki.equals(convertSimpleWareki2(seireki)))
convertSeireki2	条件38	NumberFormatException e
getSeirekiForWareki2	処理1	seireki = String.valueOf(18670000 + date)
getSeirekiForWareki2	処理2	seireki = String.valueOf(19110000 + date)
getSeirekiForWareki2	処理3	seireki = String.valueOf(19250000 + date)
getSeirekiForWareki2	処理4	seireki = String.valueOf(19880000 + date)
getSeirekiForWareki2	処理5	seireki = String.valueOf(20180000 + date)
getSeirekiForWareki2	処理6	seireki = ""
getSeirekiForWareki2	処理7	String seireki
getSeirekiForWareki2	条件8	date < 10000
getSeirekiForWareki2	処理9	int date = Integer.parseInt(nen + tuki + niti)
getSeirekiForWareki2	処理10	String niti = wareki.substring(8,10)
getSeirekiForWareki2	処理11	String tuki = wareki.substring(5,7)
getSeirekiForWareki2	処理12	String nen = wareki.substring(2,4)
getSeirekiForWareki2	処理13	String gengo = wareki.substring(0,2)
getSeirekiForWareki2	処理14	String gengo = wareki.substring(0,2)
getSeirekiForWareki2	処理15	String nen = wareki.substring(2,4)

イベント	参照番号	原本条件・処理
getSeirekiForWareki2	処理16	String tuki = wareki.substring(5,7)
getSeirekiForWareki2	処理17	String niti = wareki.substring(8,10)
getSeirekiForWareki2	処理18	int date = Integer.parseInt(nen + tuki + niti)
getSeirekiForWareki2	条件19	date < 10000
getSeirekiForWareki2	処理20	String seireki
getSeirekiForWareki2	処理21	seireki = String.valueOf(18670000 + date)
getSeirekiForWareki2	処理22	seireki = String.valueOf(19110000 + date)
getSeirekiForWareki2	処理23	seireki = String.valueOf(19250000 + date)
getSeirekiForWareki2	処理24	seireki = String.valueOf(19880000 + date)
getSeirekiForWareki2	処理25	seireki = String.valueOf(20180000 + date)
getSeirekiForWareki2	処理26	seireki = ""
getSeirekiForWareki2	条件27	date < 10000
getSeirekiForWareki2	条件28	NOT (date < 10000)
getSeirekiForWareki2	条件29	NumberFormatException e
getSimpleGengoWareki2	処理1	sbDate=new StringBuffer("")
getSimpleGengoWareki2	処理2	sbDate.replace(0,2,"R")
getSimpleGengoWareki2	処理3	sbDate.replace(0,2,"H")
getSimpleGengoWareki2	処理4	sbDate.replace(0,2,"S")
getSimpleGengoWareki2	処理5	sbDate.replace(0,2,"T")

イベント	参照番号	原本条件・処理
getSimpleGengoWareki2	処理6	sbDate.replace(0,2,"M")
getSimpleGengoWareki2	処理7	sbDate=new StringBuffer(get49Wareki2(eDate))
getSimpleGengoWareki2	処理8	StringBuffer sbDate
getSimpleGengoWareki2	処理9	StringBuffer sbDate
getSimpleGengoWareki2	処理10	sbDate=new StringBuffer("")
getSimpleGengoWareki2	処理11	sbDate=new StringBuffer(get49Wareki2(eDate))
getSimpleGengoWareki2	処理12	sbDate.replace(0,2,"R")
getSimpleGengoWareki2	処理13	sbDate.replace(0,2,"H")
getSimpleGengoWareki2	処理14	sbDate.replace(0,2,"S")
getSimpleGengoWareki2	処理15	sbDate.replace(0,2,"T")
getSimpleGengoWareki2	処理16	sbDate.replace(0,2,"M")
get49Wareki2	処理1	sbDate=new StringBuffer(getWareki2(edit49Date(eDate)))
get49Wareki2	処理2	sbDate=new StringBuffer("dateError in CifComn")
get49Wareki2	条件3	eDate.substring(4,5).equals("-")&eDate.substring(7,8).equals("-")
get49Wareki2	処理4	sbDate=new StringBuffer(getWareki2(eDate))
get49Wareki2	処理5	StringBuffer sbDate
get49Wareki2	処理6	StringBuffer sbDate
get49Wareki2	条件7	eDate.substring(4,5).equals("-")&eDate.substring(7,8).equals("-")
get49Wareki2	処理8	sbDate=new StringBuffer(getWareki2(edit49Date(eDate)))

イベント	参照番号	原本条件・処理
get49Wareki2	処理9	sbDate=new StringBuffer("dateError in CifComn")
get49Wareki2	処理10	sbDate=new StringBuffer(getWareki2(eDate))
getWareki2	処理1	wareki = wareki + "0"
getWareki2	条件2	year < 10
getWareki2	処理3	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable2[i][1].substring(0,4)) + 1
getWareki2	処理4	wareki = gengoTable2[i][0]
getWareki2	処理5	String wareki = ""
getWareki2	条件6	; i < gengoTable2.length;i++
getWareki2	処理7	break
getWareki2	条件8	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable2[i][1])
getWareki2	条件9	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable2[i][2])
getWareki2	処理10	int i = 0
getWareki2	条件11	!checkDate(seireki)
getWareki2	処理12	int wk = Integer.parseInt(seireki)
getWareki2	処理13	seireki=sbDate.toString()
getWareki2	処理14	StringBuffer sbDate=new StringBuffer(seireki)
getWareki2	条件15	seireki.substring(4,5).equals("-")
getWareki2	条件16	seireki.substring(7,8).equals("-")
getWareki2	条件17	seireki.substring(4,5).equals("-")

イベント	参照番号	原本条件・処理
getWareki2	条件18	seireki.substring(7,8).equals("-")
getWareki2	処理19	StringBuffer sbDate=new StringBuffer(seireki)
getWareki2	処理20	seireki=sbDate.toString()
getWareki2	処理21	int wk = Integer.parseInt(seireki)
getWareki2	条件22	!checkDate(seireki)
getWareki2	処理23	int i = 0
getWareki2	条件24	; i < gengoTable2.length;i++
getWareki2	条件25	Integer.parseInt(seireki) >= Integer.parseInt(gengoTable2[i][1])
getWareki2	条件26	Integer.parseInt(seireki) <= Integer.parseInt(gengoTable2[i][2])
getWareki2	処理27	break
getWareki2	処理28	String wareki = ""
getWareki2	処理29	wareki = gengoTable2[i][0]
getWareki2	処理30	int year =Integer.parseInt(seireki.substring(0,4)) - Integer.parseInt(gengoTable2[i][1].substring(0,4)) + 1
getWareki2	条件31	year < 10
getWareki2	処理32	wareki = wareki + "0"
getWareki2	条件33	NOT (seireki.substring(4,5).equals("-")&&seireki.substring(7,8).equals("-"))
getWareki2	条件34	NumberFormatException nfe
getWareki2	条件35	!checkDate(seireki)

イベント	参照番号	原本条件・処理
getWareki2	条件36	NOT (!checkDate(seireki))
getWareki2	条件37	NOT (!checkDate(seireki))
getSeireki2	処理1	boolean check = checkDate(seirekiYear,seirekiMonth,seirekiDay)
getSeireki2	処理2	seirekiDay = Integer.parseInt(warekiDay)
getSeireki2	処理3	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理4	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki2	処理5	seirekiYear = 2018 + Integer.parseInt(warekiYear)
getSeireki2	処理6	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理7	seirekiYear = 1925 + Integer.parseInt(warekiYear)
getSeireki2	処理8	seirekiYear = 1911 + Integer.parseInt(warekiYear)
getSeireki2	処理9	seirekiYear = 1867 + Integer.parseInt(warekiYear)
getSeireki2	処理10	seirekiYear = 2018 + Integer.parseInt(warekiYear)
getSeireki2	条件11	gengo.equals("M")
getSeireki2	条件12	gengo.equals("T")
getSeireki2	条件13	gengo.equals("S")
getSeireki2	条件14	gengo.equals("H")
getSeireki2	条件15	gengo.equals("R")
getSeireki2	処理16	warekiDay =wareki.substring(5,7)
getSeireki2	処理17	warekiMonth =wareki.substring(3,5)

イベント	参照番号	原本条件・処理
getSeireki2	処理18	warekiYear =wareki.substring(1,3)
getSeireki2	処理19	gengo =wareki.substring(0,1)
getSeireki2	処理20	seirekiDay = Integer.parseInt(warekiDay)
getSeireki2	処理21	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理22	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki2	処理23	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理24	warekiDay = wareki.substring(4,6)
getSeireki2	処理25	warekiMonth = wareki.substring(2,4)
getSeireki2	処理26	warekiYear = wareki.substring(0,2)
getSeireki2	処理27	seirekiDay = 10
getSeireki2	処理28	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理29	seirekiDate = Integer.toString(seirekiYear) + warekiMonth
getSeireki2	処理30	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理31	warekiMonth = wareki.substring(2,4)
getSeireki2	処理32	warekiYear = wareki.substring(0,2)
getSeireki2	処理33	seirekiDay = 10
getSeireki2	処理34	seirekiMonth = 1
getSeireki2	処理35	seirekiDate = Integer.toString(seirekiYear)
getSeireki2	処理36	seirekiYear = 1988 + Integer.parseInt(warekiYear)

イベント	参照番号	原本条件・処理
getSeireki2	処理37	warekiYear = wareki.substring(0,2)
getSeireki2	処理38	int wk = Integer.parseInt(wareki)
getSeireki2	処理39	int seirekiDay
getSeireki2	処理40	int seirekiMonth
getSeireki2	処理41	int seirekiYear
getSeireki2	処理42	String warekiDay = ""
getSeireki2	処理43	String warekiMonth = ""
getSeireki2	処理44	String warekiYear = ""
getSeireki2	処理45	String gengo=""
getSeireki2	処理46	String seirekiDate = ""
getSeireki2	処理47	String seirekiDate = ""
getSeireki2	処理48	String gengo=""
getSeireki2	処理49	String warekiYear = ""
getSeireki2	処理50	String warekiMonth = ""
getSeireki2	処理51	String warekiDay = ""
getSeireki2	処理52	int seirekiYear
getSeireki2	処理53	int seirekiMonth
getSeireki2	処理54	int seirekiDay
getSeireki2	処理55	gengo =wareki.substring(0,1)

イベント	参照番号	原本条件・処理
getSeireki2	処理56	warekiYear =wareki.substring(1,3)
getSeireki2	処理57	warekiMonth =wareki.substring(3,5)
getSeireki2	処理58	warekiDay =wareki.substring(5,7)
getSeireki2	条件59	gengo.equals("R")
getSeireki2	処理60	seirekiYear = 2018 + Integer.parseInt(warekiYear)
getSeireki2	条件61	gengo.equals("H")
getSeireki2	処理62	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	条件63	gengo.equals("S")
getSeireki2	処理64	seirekiYear = 1925 + Integer.parseInt(warekiYear)
getSeireki2	条件65	gengo.equals("T")
getSeireki2	処理66	seirekiYear = 1911 + Integer.parseInt(warekiYear)
getSeireki2	条件67	gengo.equals("M")
getSeireki2	処理68	seirekiYear = 1867 + Integer.parseInt(warekiYear)
getSeireki2	処理69	seirekiYear = 2018 + Integer.parseInt(warekiYear)
getSeireki2	処理70	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki2	処理71	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理72	seirekiDay = Integer.parseInt(warekiDay)
getSeireki2	処理73	int wk = Integer.parseInt(wareki)
getSeireki2	処理74	warekiYear = wareki.substring(0,2)

イベント	参照番号	原本条件・処理
getSeireki2	処理75	warekiMonth = wareki.substring(2,4)
getSeireki2	処理76	warekiDay = wareki.substring(4,6)
getSeireki2	処理77	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理78	seirekiDate = Integer.toString(seirekiYear) + warekiMonth + warekiDay
getSeireki2	処理79	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理80	seirekiDay = Integer.parseInt(warekiDay)
getSeireki2	処理81	warekiYear = wareki.substring(0,2)
getSeireki2	処理82	warekiMonth = wareki.substring(2,4)
getSeireki2	処理83	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理84	seirekiDate = Integer.toString(seirekiYear) + warekiMonth
getSeireki2	処理85	seirekiMonth = Integer.parseInt(warekiMonth)
getSeireki2	処理86	seirekiDay = 10
getSeireki2	処理87	warekiYear = wareki.substring(0,2)
getSeireki2	処理88	seirekiYear = 1988 + Integer.parseInt(warekiYear)
getSeireki2	処理89	seirekiDate = Integer.toString(seirekiYear)
getSeireki2	処理90	seirekiMonth = 1
getSeireki2	処理91	seirekiDay = 10
getSeireki2	処理92	boolean check = checkDate(seirekiYear,seirekiMonth,seirekiDay)
getSeireki2	条件93	NumberFormatException nfe

イベント	参照番号	原本条件・処理
checkWarekiDate2	処理1	status=true
checkWarekiDate2	条件2	!getSeireki2(wareki).equals("PARAMERR")
checkWarekiDate2	処理3	int wk=Integer.parseInt(date)
checkWarekiDate2	処理4	date=new String(wareki)
checkWarekiDate2	処理5	date=new String(wareki.substring(1,7))
checkWarekiDate2	条件6	gengo.equals("R")|gengo.equals("H")|gengo.equals("S")| gengo.equals("T")|gengo.equals("M")
checkWarekiDate2	処理7	String gengo=new String(wareki.substring(0,1))
checkWarekiDate2	処理8	String date
checkWarekiDate2	処理9	boolean status=false
checkWarekiDate2	処理10	boolean status=false
checkWarekiDate2	処理11	String date
checkWarekiDate2	処理12	date=new String(wareki)
checkWarekiDate2	処理13	String gengo=new String(wareki.substring(0,1))
checkWarekiDate2	条件14	gengo.equals("R")|gengo.equals("H")|gengo.equals("S")| gengo.equals("T")|gengo.equals("M")
checkWarekiDate2	処理15	date=new String(wareki.substring(1,7))
checkWarekiDate2	処理16	int wk=Integer.parseInt(date)
checkWarekiDate2	条件17	!getSeireki2(wareki).equals("PARAMERR")
checkWarekiDate2	処理18	status=true

イベント	参照番号	原本条件・処理
checkWarekiDate2	条件19	NOT (gengo.equals("R")|gengo.equals("H")|gengo.equals("S")|gengo.equals("T")|gengo.equals("M"))
checkWarekiDate2	条件20	NumberFormatException nfe
editSimpleWarekiDot	処理1	sb.append(wareki.substring(5,7))
editSimpleWarekiDot	処理2	sb.append(".")
editSimpleWarekiDot	処理3	sb.append(wareki.substring(3,5))
editSimpleWarekiDot	処理4	sb.append(".")
editSimpleWarekiDot	処理5	sb.append(wareki.substring(0,3))
editSimpleWarekiDot	処理6	StringBuffer sb = new StringBuffer()
editSimpleWarekiDot	処理7	StringBuffer sb = new StringBuffer()
editSimpleWarekiDot	処理8	sb.append(wareki.substring(0,3))
editSimpleWarekiDot	処理9	sb.append(".")
editSimpleWarekiDot	処理10	sb.append(wareki.substring(3,5))
editSimpleWarekiDot	処理11	sb.append(".")
editSimpleWarekiDot	処理12	sb.append(wareki.substring(5,7))